BLOCKCHAIN-SECURED SIM SWAP FRAUD DETECTION SYSTEM

AN EXPLAINABLE-AI AND SMART-CONTRACT FRAMEWORK FOR TAMPER-EVIDENT SIM REPLACEMENT DECISIONS

24IT986 – Capstone Design Level II Project

Submitted by

Krish Mehta	**111724203048**
Yukesh J	**111724203124**

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY (HONORS)
IN
INFORMATION TECHNOLOGY



R.M.K. ENGINEERING COLLEGE
(An Autonomous Institution)
R.S.M. Nagar, Kavaraipettai-601 206

NOVEMBER 2026

R.M.K. ENGINEERING COLLEGE

(An Autonomous Institution)

R.S.M. Nagar, Kavaraipettai-601 206

BONAFIDE CERTIFICATE

Certified that this project report **"Blockchain-Secured SIM Swap Fraud Detection System: An Explainable-AI and Smart-Contract Framework"** is the bonafide work of **KRISH MEHTA (111724203048), YUKESH J (111724203124)** who carried out the **24IT986 – Capstone Design Project – Level II Honours** under my supervision.

SIGNATURE

Dr. Sheerin Banu M, M.Tech., Ph.D.,
Professor and Head
Department of Information Technology
R.M.K. Engineering College
R.S.M. Nagar, Kavaraipettai,
Tiruvallur District – 601206.

SIGNATURE

Dr. Sheerin Banu M, M.Tech., Ph.D.,
Professor and Head, Supervisor
Department of Information Technology
R.M.K. Engineering College
R.S.M. Nagar, Kavaraipettai,
Tiruvallur District – 601206.

Submitted for the Project Viva-Voce held on **03-11-2026** at R.M.K. Engineering College, Kavaraipettai, Tiruvallur District – 601206.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We earnestly portray our sincere gratitude and regard to our beloved Chairperson **Smt. Manjula Munirathinam,** Chairman **Shri. R. S. Munirathinam,** our Vice Chairman, **Shri. R. M. Kishore** and our Director, **Shri. R. Jothi Naidu,** for the interest and affection shown towards us throughout the course.

We convey our sincere thanks to our Principal, **Dr. K. A. Mohamed Junaid,** for being the source of inspiration in this college.

We reveal our sincere thanks to our Project Coordinator and Professor and Head of the Department, Information Technology, **Dr. Sheerin Banu M,** for the commendable support and encouragement for the completion of our project.

We are thankful to the faculty of the Departments of Computer Science and Engineering and of Electronics and Communication Engineering for their guidance on the data-science and edge-hardware parts of this multidisciplinary project.

We take this opportunity to extend our thanks to all faculty members of the Department of Information Technology, parents and friends for all that they meant to us during the crucial times of the completion of our project.

ABSTRACT

SIM swap fraud moves a victim's mobile number to an attacker's SIM, giving the attacker the one-time passwords that protect banking and e-mail accounts. Operator checklists fail against social engineering and insider collusion, and machine-learning detectors return unexplained scores that insiders can later edit out of the log.

SIMGuard combines three layers. At the edge, a SIM800L-based gateway signs handset and radio telemetry with HMAC-SHA256. In the data-science layer, XGBoost scores each request and TreeSHAP explains the score. In the governance layer, a FastAPI risk engine merges the probability with deterministic rules and an IMEI blacklist into three tiers (auto-approve, manual review, auto-block) and anchors SHA-256 hashes of the decision and of its explanation in a Solidity smart contract.

On 4,000 held-out synthetic requests XGBoost reaches ROC-AUC 0.948 (static checklist 0.750); the engine sends 84.5% of fraud to review or block against 39.5% for the checklist, reviewing 6.8% of requests. The audit detected 577 of 577 tampering attempts, whereas a hash chain an insider can recompute detected none. A decision takes a median of 21 ms (143 ms with the blockchain write on a local chain) and about 172,240 gas to anchor.

KEYWORDS: SIM swap fraud, XGBoost, TreeSHAP, Explainable AI, Ethereum smart contract, Solidity, risk engine, HMAC, edge telemetry, tamper-evident audit trail.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
	ABSTRACT	i
	LIST OF TABLES	iv
	LIST OF FIGURES	v
1	**INTRODUCTION**	1
	1.1 Problem Statement	2
	1.2 Project Scope and Objectives	3
2	**OVERALL DESCRIPTION**	6
	2.1 Project Specification	6
	2.2 Use Case Diagram	12
	2.3 Design	14
	2.4 System Architecture	23
3	**EXTERNAL INTERFACE REQUIREMENTS**	27
	3.1 User Interfaces	27
	3.2 Hardware Interfaces	28
	3.3 Software Interfaces	29
4	**TESTING**	32
	4.1 Test Plan	32
	4.2 Test Procedure	34
	4.3 Test Results	35
5	**FUTURE ENHANCEMENTS**	47
6	**CONCLUSION**	49
	REFERENCES	51
	SAMPLE CODING	54

CHAPTER	TITLE	PAGE NO
	SCREENSHOTS	65

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE
Table 2.1	Related work and the gap it leaves	8
Table 2.2	Existing practice compared with the proposed system	11
Table 2.3	Functional requirements	11
Table 2.4	Non-functional requirements and measured values	12
Table 2.5	Actors and their responsibilities	13
Table 2.6	The 16 model features and where they come from	16
Table 2.7	Fraud archetypes in the synthetic dataset	16
Table 2.8	Deterministic security rules	18
Table 2.9	Smart contract interface	21
Table 2.10	Telemetry integrity checks	22
Table 2.11	Threat model	26
Table 3.1	Hardware interfaces	29
Table 3.2	Software stack	30
Table 3.3	REST API endpoints	30
Table 3.4	AT commands used by the edge gateway	31
Table 4.1	Test levels	34
Table 4.2	Model comparison on the held-out test set	35
Table 4.3	Three-tier policy and ablations on the test set	38
Table 4.4	Processing time per decision (n = 300, local chain)	42
Table 4.5	Gas measured on the local chain	43
Table 4.6	Detection of tampering with the off-chain store	44
Table 4.7	Edge telemetry attacks	44
Table 4.8	Automated tests	45

LIST OF FIGURES

FIG. NO	FIGURE NAME	PAGE
Fig. 2.1	Division of work across the ECE, CSE and IT modules	10
Fig. 2.2	Use case diagram	13
Fig. 2.3	Data flow diagram	14
Fig. 2.4	Three-tier decision logic of the risk engine	19
Fig. 2.5	What is hashed, what is anchored, and how the audit compares them	20
Fig. 2.6	Entity-relationship diagram of the off-chain store and the on-chain structures	23
Fig. 2.7	System architecture of SIMGuard	24
Fig. 2.8	Sequence of one SIM replacement evaluation, including manual review	25
Fig. 4.1	ROC and precision-recall curves of the five models (test set)	36
Fig. 4.2	Confusion matrix of XGBoost (test set)	36
Fig. 4.3	Global feature importance (mean |SHAP|, test set)	37
Fig. 4.4	SHAP summary: direction and size of each feature's effect	38
Fig. 4.5	Local explanation of one fraudulent request	38
Fig. 4.6	Fraud caught, analyst workload and false blocks for the four policies	40
Fig. 4.7	Decision by true class under the hybrid risk engine	40
Fig. 4.8	Detection by attack archetype (test set)	41
Fig. 4.9	Analyst workload against fraud caught as the review threshold varies	42
Fig. 4.10	Where the time goes in one decision	43
Fig. A.1	Operations console, Evaluate request view: a remote social-engineering attack is auto-blocked	65
Fig. A.2	Request with signed edge telemetry (left) and blockchain anchor with processing time (right)	65
Fig. A.3	Decision panel: genuine upgrade, auto-approved	66
Fig. A.4	Decision panel: suspicious request routed to manual review	66
Fig. A.5	Decision panel: remote social-engineering attack, auto-blocked	67
Fig. A.6	Decision panel: blacklisted handset forces auto-block	67

FIG. NO	FIGURE NAME	PAGE
Fig. A.7	Decision log with the Resolve action for manual reviews	68
Fig. A.8	Analyst resolves a manual review; the outcome is recorded on-chain	68
Fig. A.9	Blockchain ledger: contract details, counters and latest entries	69
Fig. A.10	Integrity audit passed; four telemetry attacks rejected	69
Fig. A.11	Integrity audit exposes an insider's edit of a stored decision	70
Fig. A.12	Model comparison and operating policy	70
Fig. A.13	Hardware prototype of the agent-verification terminal	71

CHAPTER 1
INTRODUCTION

The mobile phone number has quietly become the root of a person's digital identity. It is the address to which banks send transaction alerts, the channel through which one-time passwords (OTPs) are delivered for payments and logins, and the recovery route for e-mail, messaging and crypto-asset accounts. Whoever controls the number can therefore, in practice, become the account holder.

A **SIM swap** is the fraudulent transfer of a subscriber's number to a new Subscriber Identity Module (SIM) held by an attacker. Three routes dominate: social engineering of a call-centre or online-support agent, collusion with an insider at an operator or retailer, and forged identity documents presented at a store. Once the swap succeeds, calls and text messages reach the attacker, the victim's handset loses service, and every account protected only by an SMS code is exposed, usually within minutes and before the victim understands why the signal has gone.

The scale is documented even though consolidated statistics are scarce. The FBI's Internet Crime Complaint Center received 320 SIM swapping complaints with adjusted losses of about $12 million between January 2018 and December 2020, and 1,611 complaints with adjusted losses above $68 million in 2021 alone [8]. These are United States complaints only and understate the problem, because many victims never report. An empirical study of the authentication used for SIM changes at five prepaid carriers found that every one of them relied on challenges an attacker could subvert [11].

The defences in use are weak in a specific way. Manual verification places a single human in the path of every request, so one deceived or complicit agent defeats it. Static checklists are cheap but blunt: in the evaluation reported in Chapter 4, a six-signal checklist of the kind a verification desk applies catches

only 39.5% of fraudulent requests. Machine-learning detectors catch more, but they reduce a request to a score, give an analyst no reason to trust or overrule it, and write the outcome to a database that insiders with write access can later alter. The same people who can collude in a swap can quietly edit the record of it.

This project addresses the whole chain rather than one link. **SIMGuard** is a multidisciplinary framework in which the Electronics and Communication (ECE) module produces authenticated handset and radio telemetry at the edge, the Computer Science (CSE) module scores each SIM replacement request with an explainable model, and the Information Technology (IT) module turns the score into a three-tier decision (auto-approve, manual review or auto-block), explains it in plain language, and anchors a cryptographic fingerprint of both the decision and its explanation in an Ethereum smart contract so that neither can be changed afterwards without detection.

This report documents the design, implementation and measured behaviour of the prototype. Real SIM-swap records are confidential, so the model is trained on a synthetic dataset; the edge gateway is a software emulation of a SIM800L modem; and the blockchain is a local Ethereum node. Each result in Chapter 4 is stated together with what it does and does not demonstrate.

1.1 PROBLEM STATEMENT

Telecommunication operators cannot currently verify, quickly and defensibly, that a request to replace the SIM behind a phone number comes from the genuine subscriber. The problem has four connected parts.

First, the detection problem. Fraudulent requests look like legitimate ones in most respects. They differ in subtle, combined signals: the old SIM is still active on the network, the new handset has never been seen before, the request comes from a remote channel without biometric verification, or one retail agent is suddenly processing far more swaps than usual. No single signal is decisive, and a rule written for one signal produces either missed fraud or customer friction.

Second, the decision problem. A binary fraud flag is not an operational outcome. An operator must be able to approve routine requests without friction, escalate ambiguous ones to a human analyst within a finite review capacity, and stop clear attacks automatically. Some conditions, such as a handset on the stolen-device register, must override any model output.

Third, the explanation problem. An analyst who is told only that a request scored 0.87 cannot decide whether to call the customer, and an operator who blocks a customer must be able to say why. Data-protection law is moving in the same direction: the EU General Data Protection Regulation restricts decisions based solely on automated processing and requires safeguards around them [7], and India's Digital Personal Data Protection Act, 2023 obliges data fiduciaries to protect personal data with reasonable security safeguards [9]. Explanations are most useful when they are generated for every decision, in terms an analyst can read.

Fourth, the integrity problem. Decision logs held in the operator's own database are only as trustworthy as everyone who can write to that database. An insider who helped an attacker can delete the record, flip a blocked decision to approved, or rewrite the explanation to hide the signals that should have raised an alarm. Hash chains stored in the same system do not help, because the insider can recompute every hash after the edit. Integrity needs an anchor that the operator cannot rewrite.

1.2 PROJECT SCOPE AND OBJECTIVES

1.2.1 PROJECT SCOPE

The project covers the design, implementation and evaluation of an end-to-end prototype for SIM replacement decisions: the acquisition and authentication of edge telemetry; feature construction; model training with baselines; explanation of every decision; the rule-and-policy layer that produces the three-tier outcome; the smart contract and the client that anchors decisions; the off-

chain store; an audit procedure that verifies stored records against the chain; a REST interface; and an operations console for analysts and auditors.

The scope is deliberately bounded. The dataset is synthetic and generated from an explicit attack model, so absolute accuracy figures describe the prototype and not a live operator's traffic. The edge gateway is emulated in software with the same AT-command dialogue, signing and verification code that a hardware gateway would run; no physical radio measurements are reported. The blockchain is a local Ganache chain, so gas figures are measured but confirmation times are not representative of a public network. Integration with a real operator's billing, CRM or SIM-provisioning systems is outside the scope.

1.2.2 PROJECT OBJECTIVES

The primary objective is to build and evaluate a decentralised-audit, explainable decision system for SIM replacement requests. The specific objectives are:

1. **Realistic synthetic data.** To generate a labelled dataset of SIM replacement requests from an explicit attack model with four attack archetypes, legitimate hard cases such as eSIM upgrades and travellers, and a share of fraud that is undetectable from the available signals, so that results are not inflated by trivially separable classes.
2. **Authenticated edge telemetry.** To define and verify a signed packet format for handset and radio parameters that rejects altered, replayed, stale, mis-keyed and implausible packets.
3. **Accurate, calibrated detection.** To train an XGBoost classifier and compare it with a static checklist, logistic regression, a random forest and an isolation forest on a strictly held-out test set, with every threshold chosen on a separate validation set.
4. **Explanation of every decision.** To compute exact TreeSHAP contributions for each request and render the leading ones as plain-language reasons.
5. **Hybrid, governed decisions.** To combine the model probability with deterministic rules and an IMEI blacklist into three tiers, calibrating the

review and block thresholds against an explicit cost and analyst-capacity model.

6. **Tamper-evident anchoring with privacy.** To implement a Solidity contract that records a pseudonymous subscriber reference, the tier, the risk score, and SHA-256 hashes of the decision record and its SHAP explanation, with no personal data on-chain and no function that can modify or delete an entry.
7. **Verifiable audit.** To provide an audit that recomputes every stored record's hashes and compares them with the chain, and to measure how reliably it detects edits, deletions and forged insertions compared with a hash chain an insider can recompute.
8. **Operability and evidence.** To expose the system through a REST API and an operations console, to measure latency, throughput and gas, and to back the implementation with automated tests.

CHAPTER 2

OVERALL DESCRIPTION

2.1 PROJECT SPECIFICATION

2.1.1 SYSTEM STUDY

The system study examines how a SIM swap is carried out, what an operator can observe while the request is being made, and what earlier work has already done about it. Its conclusions fix the signals, the decision structure and the integrity requirements of the design.

Anatomy of a SIM swap

1. The attacker gathers personal details about the victim (name, date of birth, address, recent transactions) from data leaks, social media or phishing.
2. The attacker, or a colluding agent, asks the operator to replace the SIM behind the victim's number: by telephone or chat posing as the victim, at a retail outlet with forged documents, or through an insider who simply processes the request.
3. The operator's agent applies a verification step, typically security questions or a document check, and activates the new SIM.
4. The victim's SIM is deactivated; calls and SMS now reach the attacker's SIM.
5. The attacker triggers password resets and payment authorisations and reads the OTPs. The victim notices only when the handset shows no service.

The key observation is that steps 2 and 3 are the only point at which the swap can be stopped cheaply, and they are performed by people working through a checklist. Lee et al. showed that the challenges used at that point could be subverted at every carrier they tested [11], and the FBI notes that the schemes are growing and recommends that carriers verify customer credentials before moving numbers [8].

Signals available at the time of the request

Although the attacker can imitate a customer's answers, the request itself leaves traces that the operator already holds. These fall into three groups that the design uses directly:

- **Subscriber and account history** (billing and CRM data): account age, current SIM age, number of recent swaps, recent authentication failures and profile changes.
- **Request context**: the channel (store, call centre, online, retailer), the hour and day, whether biometric KYC was completed, and how many swaps the handling agent has processed in the last day.
- **Radio and network observations** (edge gateway and network probes): whether the old SIM is still active on the network, whether the new handset has ever been seen before, how far the request is from the subscriber's home cell, and recent cell changes. These are the signals an attacker cannot forge by answering questions, and they are the signals the edge module authenticates.

Related work

SIM swap fraud detection. Published work on detecting SIM swaps with machine learning is limited. Bhushan et al. propose an AI/ML pipeline that processes telecom call records for unusual patterns and sets thresholds for normal behaviour [2]. Sotonye et al. propose combining machine-learning anomaly detection with multi-factor authentication and report an 80% reduction in fraudulent attempts [23]; the published record does not describe the dataset, models or evaluation protocol behind that figure, so it cannot be used as a benchmark. Both works treat the output as a detection score and do not address how decisions are explained or protected from later alteration.

Gradient boosting and explanation. Gradient-boosted trees, and XGBoost in particular [5], are a strong default for tabular fraud data; random forests [3] and isolation forests [12] are common supervised and unsupervised baselines. SHAP assigns each feature a Shapley value [20], the unique attribution satisfying local accuracy, missingness and consistency [14], and TreeSHAP

computes it exactly for tree ensembles in polynomial time [13]. These give the per-decision explanations that interpretable-ML research asks for [6].

Blockchain for audit. Blockchains provide an append-only ledger whose history cannot be rewritten without the agreement of the network [15]. Ethereum adds programmable contracts that enforce rules on who may append and what may be stored [4, 25]. Machine-learning fraud detection combined with a blockchain smart contract has been proposed for transaction fraud [19], where a smart contract deploys the model to predict the nature of new transactions.

Table 2.1 – Related work and the gap it leaves

Work	Focus	Contribution	Limitation for SIM swap decisions
[8] FBI IC3	Incident statistics	Counts and losses of reported SIM swaps	US complaints only; no detection method
[11] Lee et al.	Carrier authentication	Showed SIM-change challenges are subvertible	Measures the weakness; proposes no detector
[2] Bhushan et al.	ML on call records	AI/ML pipeline with learned thresholds	No explanation or integrity mechanism
[23] Sotonye et al.	ML + MFA	Reports an 80% fewer fraud attempts	Dataset and method not described in the record
[19] Sensors 2022	ML + smart contract	Smart contract deploys the ML model for new transactions	Transaction fraud; no explanation hash; no risk layer
[13] Lundberg et al.	Explainable trees	Exact TreeSHAP for ensembles	General method; not tied to an audit trail

Gap. From the works we could examine (in several cases only their abstracts or published records), no system combines (i) authenticated radio-side telemetry, (ii) an explainable model, (iii) a deterministic rule layer that can override the model and produces an actionable three-tier outcome, and (iv) an immutable record of **both the decision and its explanation** for SIM replacement decisions. This project builds and measures exactly that combination.

2.1.2 EXISTING SYSTEM

In current practice a SIM replacement is authorised by a human agent working through a verification script: security questions, an identity document, sometimes a one-time code sent to the very number being replaced. Fraud teams

add static rules after the fact, such as flags for repeated swaps on one number or for a high-risk region. Where machine learning is used it produces a score, and the resulting decision is written to the operator's own database or log files.

The disadvantages follow directly from this arrangement:

- **A single point of failure.** One deceived or complicit agent is enough, and the checklist cannot see whether the old SIM is still live or the handset has ever been seen on the network.
- **Blunt rules.** The six-signal checklist evaluated in Chapter 4 catches 39.5% of fraudulent requests while leaving 121 of 200 fraudulent test requests approved.
- **Opaque scores.** A score without reasons cannot guide an analyst's call to the customer and cannot be explained to a regulator or an aggrieved customer.
- **Mutable evidence.** Logs held by the operator can be altered by anyone with write access, so they cannot settle a dispute between operator, customer and bank.
- **Unauthenticated inputs.** Network and handset data reach the decision logic without proof that they were not altered on the way.

2.1.3 PROPOSED SYSTEM

SIMGuard treats a SIM replacement request as a decision that must be accurate, explained and permanently attributable. The work is divided across the three departments so that each layer can be built and validated independently (Fig. 2.1).

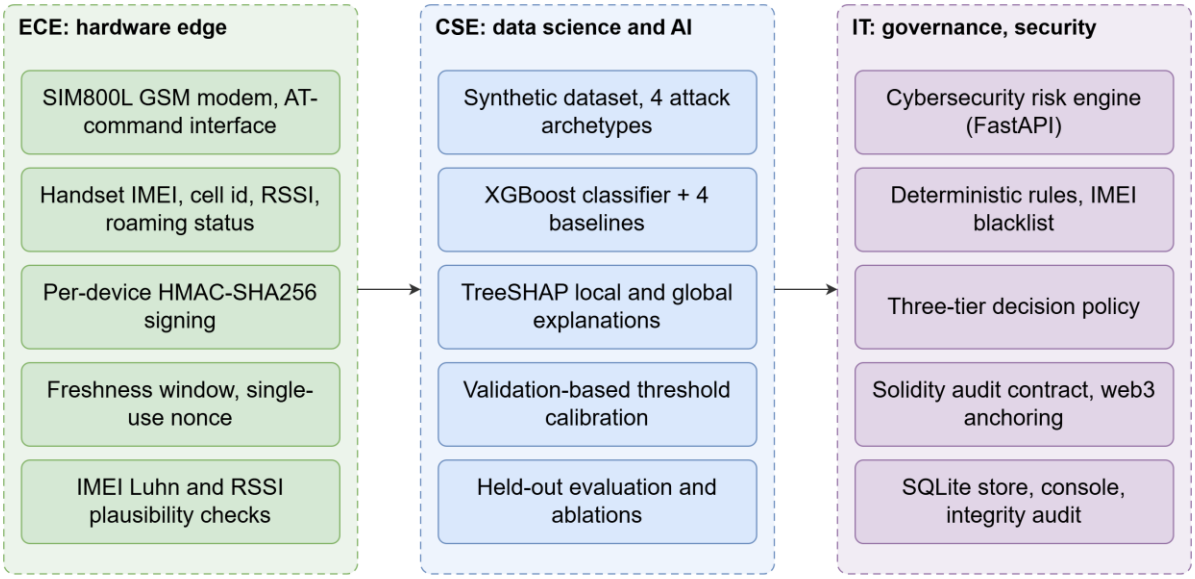


Fig. 2.1 – Division of work across the ECE, CSE and IT modules

The proposed system has five parts:

1. **Authenticated edge telemetry (ECE).** A gateway built around a SIM800L GSM modem reads the handset IMEI, serving cell and signal strength through AT commands, adds the network-side observations, and signs the packet with a per-device HMAC-SHA256 key.
2. **Explainable detection (CSE).** An XGBoost classifier turns the 16-element feature vector into a fraud probability; TreeSHAP attributes the probability to the features.
3. **Hybrid risk engine (IT).** Deterministic rules and an IMEI blacklist are combined with the probability under a calibrated policy to give one of three outcomes: auto-approve, manual review or auto-block. Critical rules override the model.
4. **Tamper-evident anchoring (IT).** The decision record and the SHAP explanation are canonicalised and hashed with SHA-256; the hashes, a pseudonymous subscriber reference, the tier and the risk score are appended to a Solidity smart contract. The full record stays off-chain.
5. **Verification and operations (IT).** An audit recomputes the hashes of every stored record and compares them with the chain; a REST API and an operations console serve analysts, administrators and auditors.

Table 2.2 – Existing practice compared with the proposed system

Aspect	Current practice	ML-only detectors (literature)	SIMGuard
Decision basis	Agent judgement and checklist	Model score	Model + deterministic rules + blacklist
Output	Approve or reject	Fraud / not fraud	Auto-approve, manual review or auto-block
Explanation	Agent's note	Usually none	TreeSHAP reasons for every decision
Audit record	Operator database	Operator database	Hashes of decision and explanation on a smart contract
Insider edits	Undetectable	Undetectable	Detected by the audit
Input integrity	None	None	HMAC-signed, fresh, single-use telemetry
Privacy of the record	Full data in the operator's system	As current practice	Pseudonymous reference; no personal data on-chain

Functional and non-functional requirements

Table 2.3 – Functional requirements

ID	Functional requirement	Verified by
FR1	Accept a SIM replacement request with signed edge telemetry and return a decision with reasons.	API tests, system test
FR2	Reject telemetry that is altered, replayed, stale, from an unknown device or implausible.	Telemetry tests, edge experiment
FR3	Score each request with the trained model and compute exact SHAP contributions.	Explanation tests (additivity)
FR4	Apply deterministic rules and the IMEI blacklist; critical rules force auto-block, high rules force at least review.	Rule tests
FR5	Return one of three tiers using thresholds calibrated on validation data.	Policy experiment
FR6	Anchor hashes of the decision and explanation on the smart contract; store no personal data on-chain.	Contract and API tests
FR7	Let an analyst resolve a manual review; record the outcome on-chain as a separate, append-only entry.	Contract and API tests
FR8	Verify every stored record against the chain and report any mismatch, deletion or unanchored record.	Tamper experiment
FR9	Provide an operations console showing request, decision, explanation, anchor and audit results.	Console screenshots

Table 2.4 – Non-functional requirements and measured values

ID	Non-functional requirement	Target	Measured
NFR1	Decision latency excluding the blockchain write	< 100 ms median	21 ms median, 59 ms p95
NFR2	End-to-end latency including the chain write (local chain)	< 1 s median	143 ms median, 393 ms p95
NFR3	Detection of tampering with off-chain records	100%	577 of 577 trials
NFR4	Rejection of doctored telemetry	100%, no false rejections	100% rejected; 100 of 100 genuine accepted
NFR5	Gas to anchor one decision	< 250,000	172,240 median
NFR6	Privacy: no MSISDN or IMEI on-chain	None stored	Pseudonymous reference only (test)
NFR7	Reproducibility	Fixed seeds, scripted pipeline	One command reproduces all results
NFR8	Consistency of the online and offline decision paths	Identical	0 mismatches in 300

2.2 USE CASE DIAGRAM

Fig. 2.2 shows how the actors interact with the system. The retail agent or call-centre staff member submits the request; the edge gateway supplies the signed telemetry that accompanies it; the risk engine evaluates, explains and anchors the decision; the fraud analyst resolves requests routed to manual review; the network administrator configures thresholds and monitors the engine; and the auditor or regulator independently verifies the audit trail.

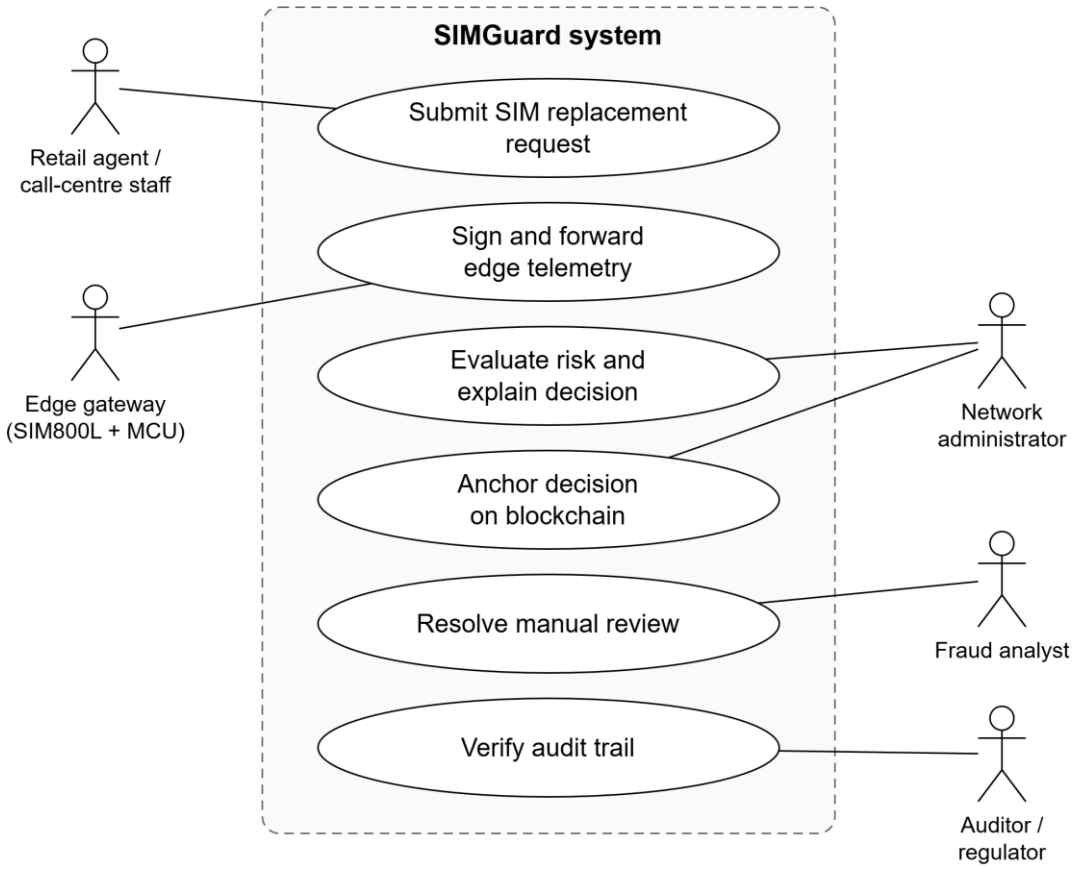


Fig. 2.2 – Use case diagram

Table 2.5 – Actors and their responsibilities

Actor	Responsibility	Main use cases
Retail agent / call-centre staff	Initiates a SIM replacement on behalf of a customer	Submit request; receive decision and reasons
Edge gateway	Reads handset and radio parameters; signs the packet	Sign and forward telemetry
Fraud analyst	Handles requests the engine routes to review	Resolve manual review (decision, justification)
Network administrator	Operates the engine and the blacklist	Monitor decisions; manage thresholds and keys
Auditor / regulator	Checks that decisions were not altered	Verify audit trail; inspect ledger

The data flow is shown in Fig. 2.3. Telemetry and request fields enter the engine (P1, P2); the model returns a probability and SHAP vector (P3); rules and the blacklist produce the decision (P4); the decision is hashed, anchored and stored (P5) and returned to the agent with its reasons; review outcomes and audit reads (P6) use the stored records and the ledger.

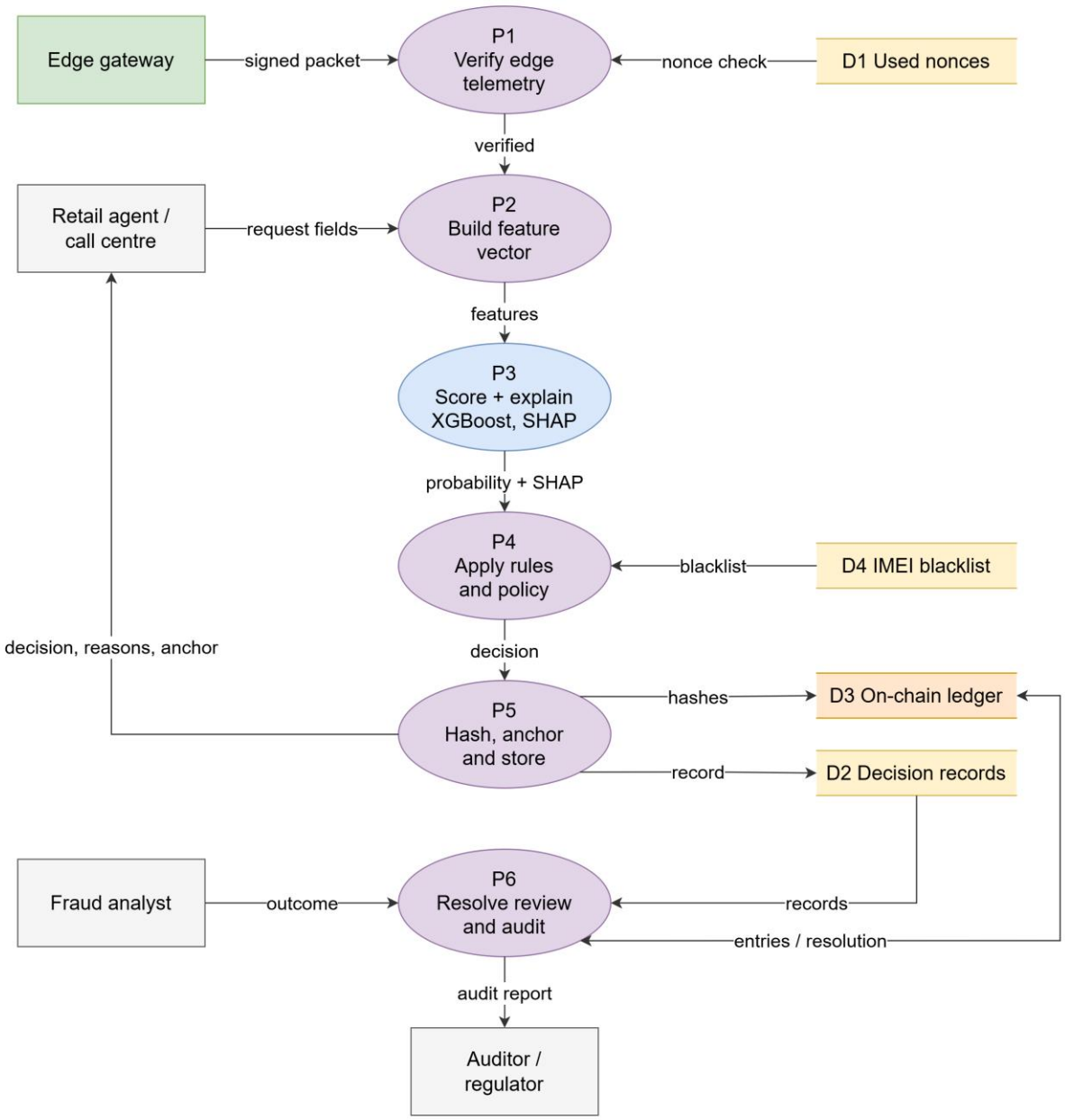


Fig. 2.3 – Data flow diagram

2.3 DESIGN

SIMGuard follows a layered, modular design so that each concern (acquisition, scoring, governance, integrity, presentation) can be developed, tested and replaced independently. Four principles guided the decisions below.

- **Never trust a single component for integrity.** The model can be wrong, the database can be edited and the network can be tampered with, so each is checked by a mechanism that does not depend on it: rules for the model, the chain for the database, signatures for the network.

- **Explain every decision.** An explanation is produced and recorded for each request, not only for those that are challenged.
- **Keep personal data off the chain.** The chain stores pseudonyms and hashes. Data that must be erasable stays in an operator database that can be erased.
- **Make measurements reproducible.** Every threshold is chosen on validation data, every random process is seeded and every reported number is generated by a script.

Dataset design

Real SIM-swap records are confidential under telecom licence conditions and under data-protection law [9], so the prototype is trained and evaluated on a synthetic dataset generated from an explicit attack model. The generator is designed to avoid the most common flaw of synthetic fraud data, in which fraud and legitimate records occupy disjoint regions so that any model scores near-perfectly. Three measures prevent this. Legitimate and fraudulent requests share the same base distributions and fraud only shifts the probability of particular signals. Legitimate **hard cases** exist: customers who upgrade to an eSIM keep the old SIM live, travellers request swaps far from home, and one request in eight is made at an unusual hour. And a fraction of fraud, about 6%, is drawn from the legitimate distribution, so it is **undetectable from these features by design**.

The dataset contains 20,000 requests of which 1,000 (5%) are fraudulent, a rate chosen to make class imbalance realistic. 128 fraudulent and 36 legitimate requests target a blacklisted handset, the latter representing second-hand devices that were reported stolen after sale. Each row carries a 16-element feature vector (Table 2.6) plus identifiers that are used only by the engine, never by the model.

Table 2.6 – The 16 model features and where they come from

Feature	Meaning	Source
account_age_days	Age of the subscriber account	BSS
sim_age_days	Age of the SIM currently in service	BSS
swaps_last_180d	SIM replacements in the previous 180 days	BSS
hour_of_request	Hour of day of the request (0-23)	BSS
is_weekend	Request made on a weekend	BSS
channel_code	Store, call centre, online or retailer	BSS
kyc_biometric_verified	Biometric KYC completed for this request	BSS
auth_failures_7d	Failed authentication attempts in 7 days	BSS
profile_change_7d	Profile details changed in the last 7 days	BSS
agent_swaps_24h	Swaps processed by the handling agent in 24 hours	BSS
imei_changed	New handset differs from the previous one	Edge (signed)
device_age_days	Days the new handset has been on the network (0 = never seen)	Edge (signed)
old_sim_last_seen_min	Minutes since the old SIM was last active	Edge (signed)
distance_from_home_km	Distance between request location and home cell	Edge (signed)
cell_changes_24h	Serving-cell changes in the last 24 hours	Edge (signed)
roaming_flag	Subscriber is roaming	Edge (signed)

Fraud is generated from four archetypes drawn from the attack routes in Section 2.1.1 (Table 2.7). Each starts from legitimate behaviour and then shifts the signals that attack characteristically changes.

Table 2.7 – Fraud archetypes in the synthetic dataset

Archetype	Mechanism	Signals shifted	Test requests
Social engineering	Remote support is deceived into swapping	Call-centre or online channel, no biometric KYC, auth failures, profile change	89
Insider (retailer)	A colluding retailer processes many swaps	Retailer channel, agent swaps in 24 h far above normal, evening hours	46
Forged ID (store)	Forged documents presented in a store	Store channel, large distance from home, profile change	37
Rapid re-swap	A compromised number is swapped again	SIM age under 3 weeks, repeated swaps	13
Stealth	Indistinguishable from a genuine request	None (drawn from the legitimate distribution)	15

All archetypes share two traits that reflect how attacks actually run: the victim's old SIM is usually still active when the request is made, and the attacker

usually presents a handset that has not been seen on the network before. Both are visible only to the radio-side telemetry, which is why that data is authenticated.

Model design

The data are split once, stratified by class and seeded, into training (12,000 requests), validation (4,000) and test (4,000) sets, each containing the same fraud rate. The test set is used only for the final report. Every decision threshold in the system, including each baseline's operating point and the risk-engine thresholds, is chosen on the validation set. Because fraud is 5% of requests, the precision-recall curve is reported alongside ROC, which can look optimistic under imbalance [18].

The proposed classifier is XGBoost [5] with 4-level trees, learning rate 0.05, row and column subsampling of 0.8 and a minimum child weight of 2. Training stops early when validation precision-recall AUC no longer improves for 50 rounds, which selected 198 trees. Four baselines are trained on the same split: the static six-signal checklist used by verification desks, logistic regression on standardised features, a random forest [3] and an isolation forest [12] as an unsupervised detector.

Explanation design

For every request, TreeSHAP [13] computes one contribution per feature, expressed in log-odds so that the base value plus the sum of the contributions equals the model's raw output exactly. XGBoost computes these values natively. The prototype verifies the identity on the held-out set: the largest absolute discrepancy is 5.7e-06, which is float rounding. The base value is -2.97 log-odds, corresponding to the 5% prevalence.

The numeric vector is turned into reasons an analyst can read. The eight contributions largest in magnitude are kept, and each is paired with a sentence describing the feature value, for example “Request made 94 km from the home cell (+3.38)” or “Biometric KYC not completed (+0.80)”. A summary sentence

assembles the leading four reasons and any rules that fired. The numeric vector and the feature values it explains are what is hashed on-chain, so the readable text can always be regenerated and checked.

Risk engine and policy design

The probability alone is not an operational decision. The risk engine adds deterministic rules, each with a severity that fixes its effect (Table 2.8). A CRITICAL rule forces auto-block whatever the model says; a HIGH rule forces at least manual review; each MEDIUM rule adds 0.10 to the risk score.

Table 2.8 – Deterministic security rules

Rule	Severity	Condition	Rationale
IMEI_BLACKLISTED	CRITICAL	New handset is on the stolen / blocked register	A reported device must never receive a swapped number
SWAP_COOLDOWN	HIGH	Current SIM issued less than 7 days ago	Back-to-back swaps are a hallmark of a compromised number
AGENT_VELOCITY	HIGH	Agent processed 12 or more swaps in 24 hours	Insider collusion processes unusual volume
ACTIVE_SIM_REMOTE_ REQUEST	HIGH	Old SIM active in the last 15 min and request more than 50 km from home	A live SIM and a distant request cannot both be the owner
WEAK_KYC_REMOTE_C HANNEL	MEDIUM	Call-centre or online request without biometric KYC	The channel most exposed to social engineering
AUTH_FAILURE_SPIKE	MEDIUM	5 or more failed authentications in 7 days	Credential guessing before the request

The risk score is min(1, p + 0.10 x m), where p is the model probability and m the number of MEDIUM rules that fired. Fig. 2.4 gives the decision logic. A request is auto-blocked if any CRITICAL rule fired or the risk reaches the block threshold; otherwise it goes to manual review if any HIGH rule fired or the risk reaches the review threshold; otherwise it is auto-approved.

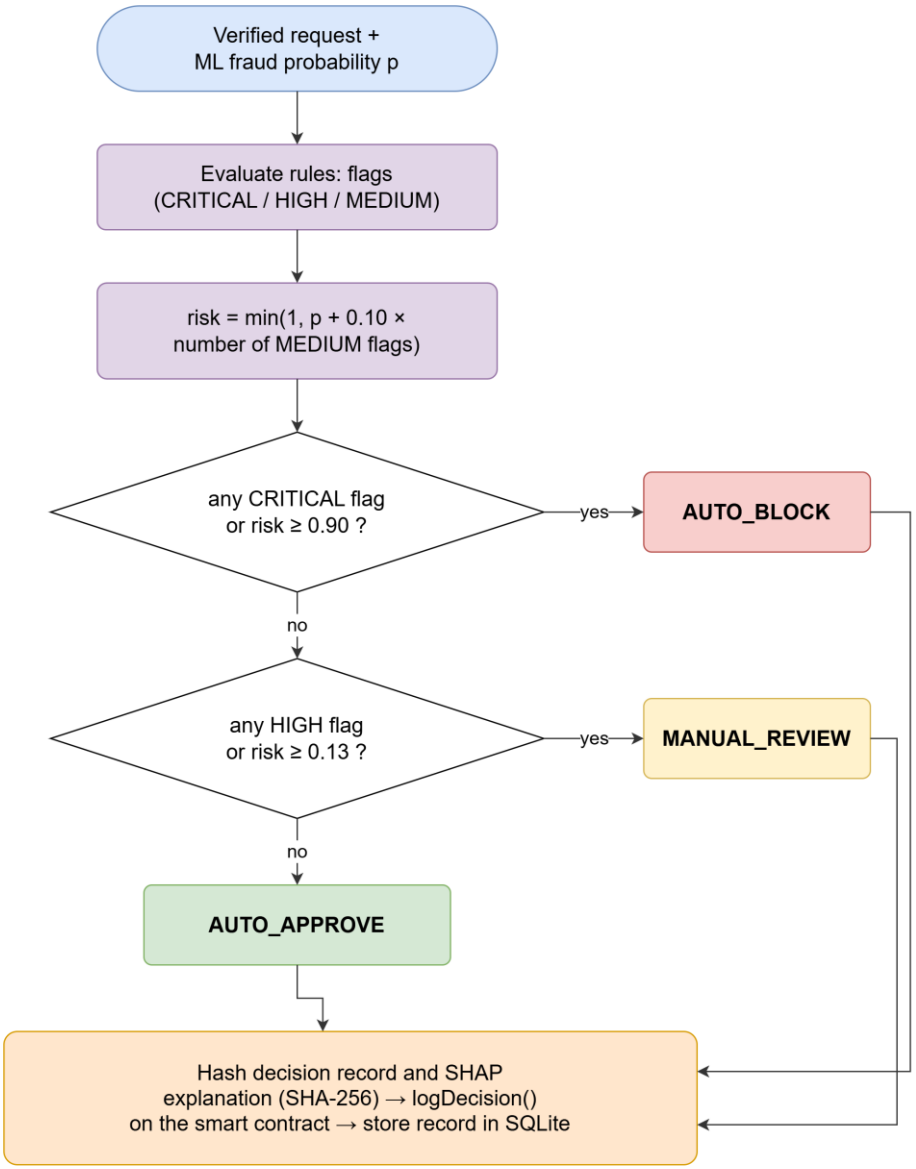


Fig. 2.4 – Three-tier decision logic of the risk engine

The two thresholds are not guessed. They are chosen on the validation set by minimising an explicit operating cost: 100 for each fraudulent request that is auto-approved, 20 for each legitimate customer auto-blocked, and 1 for each manual review, subject to a capacity limit that analysts can review at most 8% of requests. A grid search over both thresholds selected a review threshold of 0.13 and a block threshold of 0.9. The cost weights are assumptions that an operator should replace with its own figures; the method stays the same.

Blockchain design

The design question is what to put on-chain. Storing whole records would expose personal data and cost far more gas; storing nothing would leave the audit trail mutable. SIMGuard stores **fingerprints**: the chain holds SHA-256 hashes [16] of the decision record and of the SHAP explanation, together with the tier, the risk score in basis points, a timestamp, and a pseudonymous subscriber reference. The full record and the readable explanation stay in the operator's database. At audit time the hashes are recomputed from the stored records and compared with the chain (Fig. 2.5); any difference proves that the stored record changed after the decision.

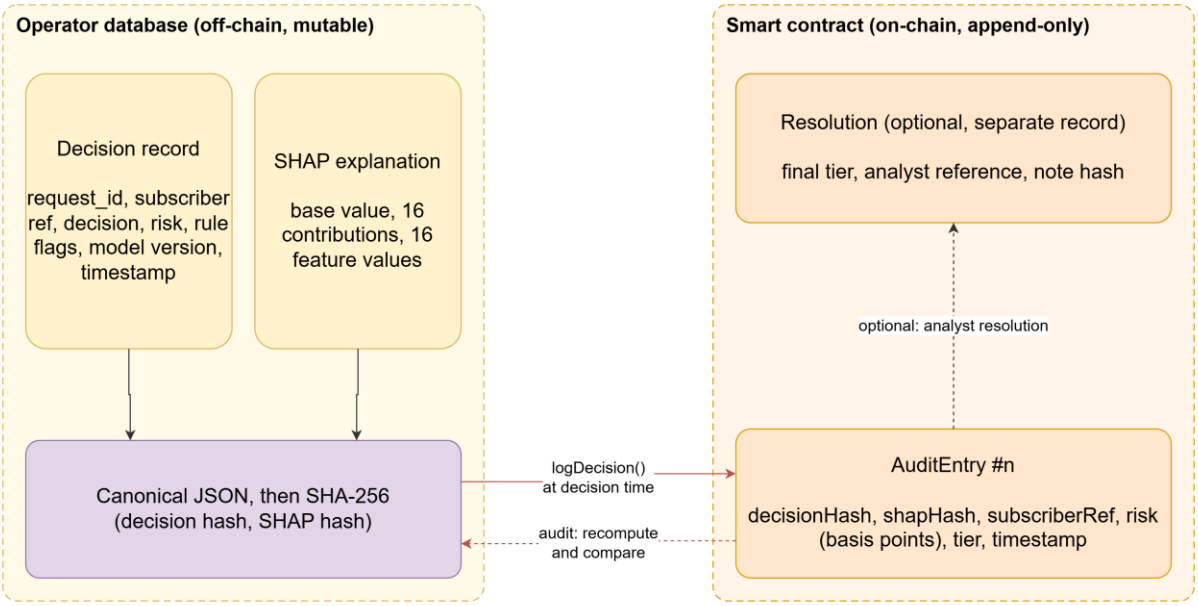


Fig. 2.5 – What is hashed, what is anchored, and how the audit compares them

Three details make the fingerprints reliable. **Canonical encoding:** records are serialised as JSON with sorted keys, no whitespace and fixed rounding, so the same record always yields the same hash. **Explanation coverage:** the SHAP hash covers the numeric contributions and the feature values they explain, so neither can be edited independently. **Pseudonymisation:** the subscriber reference is an HMAC-SHA256 of the mobile number under an operator-held key [10], so the

number cannot be recovered from the chain and the references of one subscriber remain linkable only by the key holder.

The contract, **SIMSwapAuditLog**, is written in Solidity [22] and compiled with solc 0.8.24. Its interface is summarised in Table 2.9. It has no function that updates or deletes an entry; a manual-review outcome is recorded as a separate resolution and the original entry is never touched. Only addresses authorised by the owner may append, which prevents outsiders from filling the ledger with false entries. Counters per tier are maintained on write, so reading the statistics costs constant time, unlike a loop over all entries.

Table 2.9 – Smart contract interface

Function	Access	Purpose
logDecision(ref, tier, riskBps, decisionHash, shapHash)	Authorised engine	Append an audit entry; emits DecisionLogged
resolveReview(entryId, finalTier, analystRef, noteHash)	Authorised engine	Record the analyst's outcome for a MANUAL_REVIEW entry, once
getEntry(id), getResolution(id)	Public (view)	Read an entry or its resolution
verifyEntry(id, decisionHash, shapHash)	Public (view)	Compare supplied hashes with the anchored ones
getSubscriberEntries(ref), getStats()	Public (view)	Entries of one pseudonym; counts per tier
authorizeEngine, revokeEngine, transferOwnership	Owner only	Manage which addresses may write

Edge telemetry design

The edge gateway is built around a SIM800L GSM modem controlled with AT commands [21]. It reads the handset IMEI (AT+GSN), the signal quality (AT+CSQ), the registration status (AT+CREG?) and the serving cell (AT+CENG?). Observations that a modem cannot produce alone, such as how long ago the old SIM was last active, come from a network probe and are added by the gateway. The resulting payload is signed with HMAC-SHA256 [10] using a key held only by that device, over a canonical encoding that is independent of programming language.

The engine accepts a packet only if every check in the following table passes. The checks are cheap, so they run before any model inference.

Table 2.10 – Telemetry integrity checks

Check	Attack it stops	Implementation
Registered device and valid HMAC-SHA256	Forged or altered payload; unknown device	Per-device key; constant-time comparison over the canonical packet
Freshness window	Stale captured packet	Timestamp within 120 s of the engine clock
Single-use nonce	Replay of a valid packet	Nonce stored after processing; second use rejected
IMEI checksum	Corrupted or invented handset identity	Luhn check digit of the IMEI (3GPP TS 23.003 numbering)
Plausibility	Out-of-range readings	RSSI between -120 and -30 dBm; counters not negative

The engine overrides any handset or radio value stated by the caller with the signed values, so a compromised front-end cannot feed the model false radio observations without breaking the signature.

Data design

The operator database is SQLite. The **decisions** table holds one row per request: a pseudonymous subscriber reference, a masked number, the channel, a hashed agent reference, the tier, risk and probability, the rules that fired, the feature vector, the SHAP contributions, the readable reasons, both hashes, the transaction hash, block and gas, the latency breakdown and, when applicable, the analyst's resolution. The **imei_blacklist** table holds the stolen-device register and **used_nonces** holds the nonces of processed telemetry packets. Fig. 2.6 shows the tables and how a row corresponds to exactly one on-chain entry.

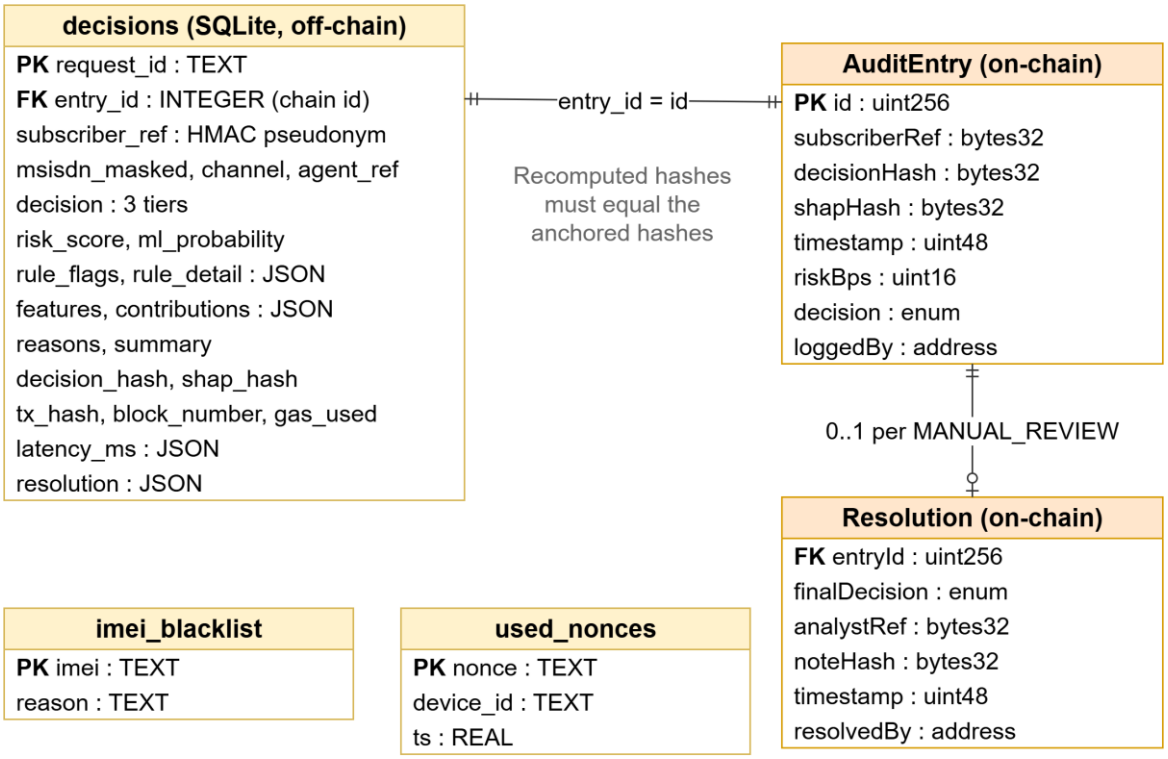


Fig. 2.6 – Entity-relationship diagram of the off-chain store and the on-chain structures

2.4 SYSTEM ARCHITECTURE

Fig. 2.7 shows the layers of SIMGuard. Signed packets flow from the edge layer into the risk engine, which is the only component that talks to the model, the database and the blockchain. The presentation layer reaches the engine only through its REST API.

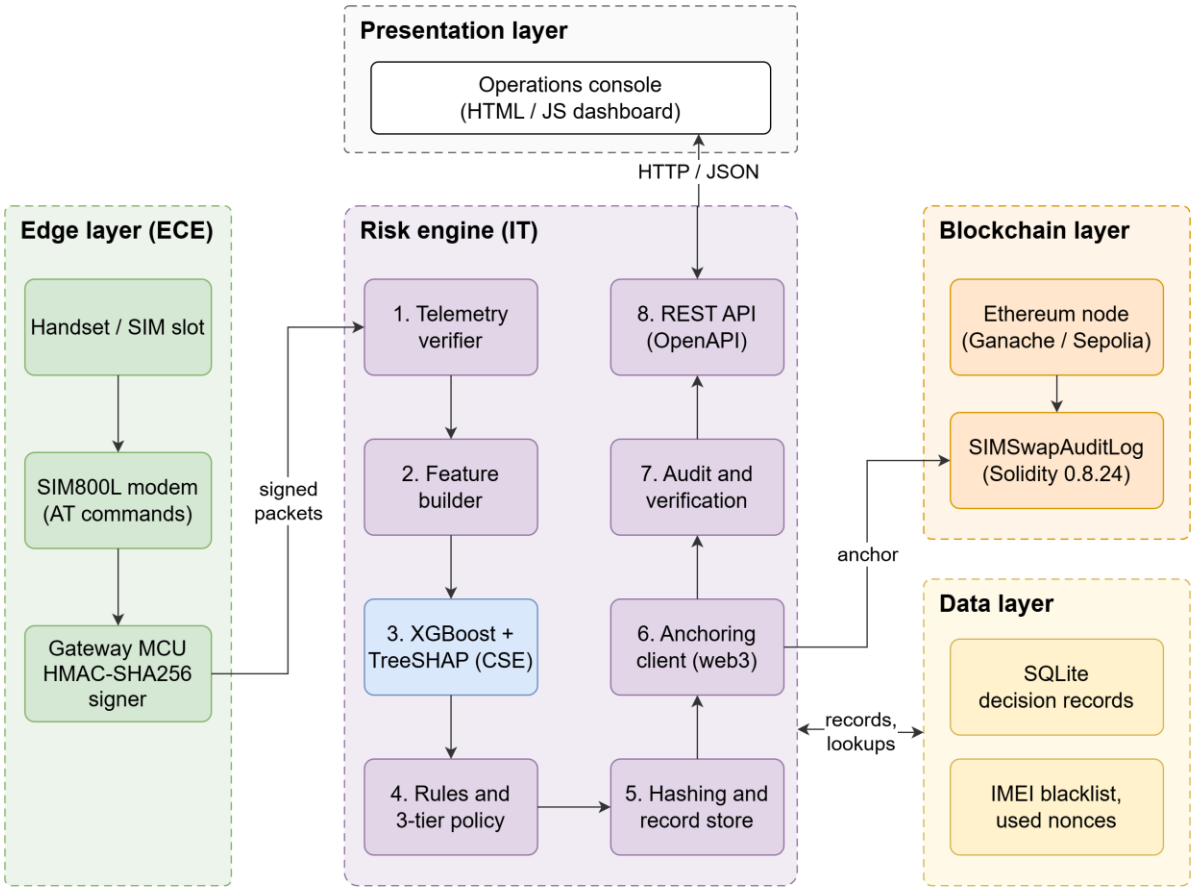


Fig. 2.7 – System architecture of SIMGuard

Edge layer (ECE). The handset and SIM slot, the SIM800L modem and a microcontroller that signs the packet. It is the only source of radio-side observations and the only holder of the device keys.

Risk engine (IT). Eight modules run in sequence: telemetry verification, feature construction, model and SHAP scoring, rules and policy, canonical hashing and record storage, the anchoring client, audit and the REST interface. The CSE model is loaded once and scores a request in a few milliseconds.

Blockchain layer. An Ethereum node hosting the audit contract. The prototype uses a local Ganache node; replacing it with a public testnet or a permissioned network changes only the node address and the signing account.

Data layer. SQLite for decision records, the IMEI blacklist and the used nonces.

Presentation layer. The operations console for analysts, administrators and auditors.

Fig. 2.8 follows one request through the system, including the review path. Steps 4 to 10 take place in memory and complete within a few milliseconds; step 11, the blockchain transaction, is the slowest step and dominates the end-to-end time (Section 4.3).

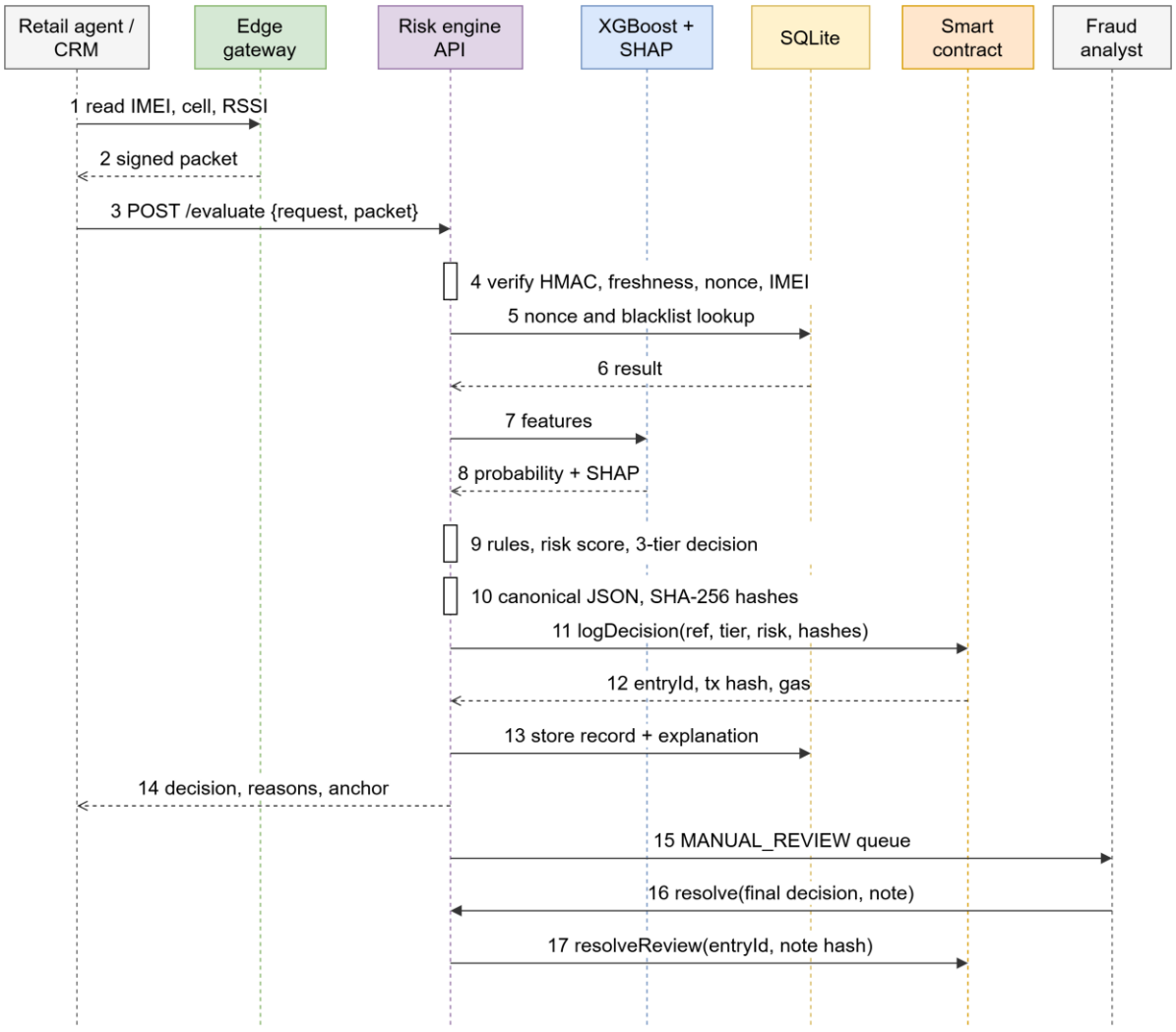


Fig. 2.8 – Sequence of one SIM replacement evaluation, including manual review

Threat model. Table 2.11 lists the threats considered, the control that addresses each and the risk that remains. Stating the residual risk is as important as the control: anchoring proves what was decided, not that the inputs were true.

Table 2.11 – Threat model

Threat	Control	Residual risk
Agent deceived by social engineering	Model, rules and manual-review tier reduce reliance on the agent	Stealth fraud that looks genuine is not detected
Insider edits or deletes a stored decision	Hash anchoring and audit expose any change to a record	An engine that never anchors a decision at all must be caught by the missing-anchor check
Telemetry altered in transit	HMAC over canonical packet	Compromise of the gateway itself
Captured packet replayed or delayed	Single-use nonce and 120 s freshness window	None beyond clock accuracy
Device key stolen	Per-device keys limit the damage to one gateway	A stolen key can forge telemetry until revoked; a secure element would help
Engine signing key stolen	Owner can revoke an engine address	Single signing account in the prototype; multi-signature advisable
Attacker mimics legitimate behaviour	Rules do not depend on the model; critical rules always block	Cannot be excluded; recall on stealth fraud is low
Personal data exposed through the chain	Pseudonymous reference, hashes only, nothing else on-chain	Linkage by the key holder; hashes of records with personal data
Chain unavailable	Decision is computed before anchoring	Prototype anchors synchronously, so the request fails; a retry queue is future work

CHAPTER 3

EXTERNAL INTERFACE REQUIREMENTS

3.1 USER INTERFACES

The system provides two user-facing interfaces. The **operations console** is a web application for analysts, administrators and auditors; it carries every function a person needs without any command-line access. The **interactive API documentation** generated from the service's OpenAPI description lets developers and integrators try each endpoint. The two are served by the same risk-engine process and share one source of truth: every value on every console screen is returned by the API from the model, the database or the smart contract, and none is computed or invented in the browser.

Evaluate request view. The analyst chooses one of four scenarios, each a real held-out request: a genuine upgrade, a suspicious request, a remote social-engineering attack and a swap onto a blacklisted handset. The left panel shows the request fields and the signed edge telemetry with its verification status. The centre panel shows the decision with an icon, label and risk score, a gauge marking the review and block thresholds, the model probability, the rules that fired, and a diverging bar chart of the leading SHAP contributions with a plain-language summary. The right panel shows the blockchain anchor (ledger entry, block, transaction hash, gas, both hashes and the subscriber reference) and a breakdown of the time taken by each processing step. The whole view is shown in Fig. A.1, the request and anchor panels in Fig. A.2 and the decision panel for each scenario in Fig. A.3, Fig. A.4, Fig. A.5 and Fig. A.6.

Decision log. A filterable table of stored decisions with the entry number, time, masked subscriber number, channel, tier, risk score, rules triggered and transaction. Requests in the manual-review tier carry a **Resolve** action that opens

a dialog for the analyst's final decision and written justification; the outcome is then recorded on-chain (Fig. A.7, Fig. A.8).

Blockchain ledger. Counters per tier read from the contract, the contract address, compiler, latest block and deployment gas, and a table of the latest on-chain entries with their hashes and any analyst outcome (Fig. A.9).

Integrity audit. An auditor can recompute every stored record's hashes and compare them with the chain. Two simulations demonstrate the protection: an insider edit of the off-chain database (change a decision, alter an explanation, delete a record), after which the audit lists exactly which entry was altered and how; and four attacks on the edge telemetry, each of which the engine rejects with a stated reason (Fig. A.10, Fig. A.11).

Model performance. The held-out comparison of the five models, the operating policy (thresholds and cost weights) and the ROC, precision-recall and feature-importance figures (Fig. A.12).

3.2 HARDWARE INTERFACES

The edge layer is designed around commodity components. In this prototype the modem and microcontroller are replaced by a software emulation that speaks the same AT-command dialogue and runs the same signing code, so the complete pipeline can be exercised and measured without hardware; the table states which interfaces are real and which are emulated.

A first physical build of the agent-side input hardware is shown in Fig. A.13. An RFID reader module (RC522) identifies the retail agent, a 4 x 4 matrix keypad takes the digits of the request, and a small OLED display reports the state of the terminal ("AGENT VERIFIED", "Enter request digits"). A microcontroller board powered over USB connects them on solderless breadboards. The photograph does not include the SIM800L modem, which remains emulated as Table 3.1 states.

Table 3.1 – Hardware interfaces

Component	Interface	Role	Status in this prototype
SIM800L GSM module	UART, AT commands; 3.4-4.4 V supply	Reads handset IMEI, signal quality, registration status and serving cell	Emulated (gateway_sim.py)
Gateway microcontroller (ESP32 / Arduino class)	UART to the modem; Wi-Fi or Ethernet to the engine	Assembles the packet, holds the device key, computes HMAC-SHA256	Signing code real; hardware not built
SIM / handset slot	ISO 7816 SIM contacts	Presents the new SIM and handset at the point of sale	Not applicable
Network probe (HLR / HSS feed)	Operator network API	Time since the old SIM was seen, distance from home cell, cell changes	Values supplied from the dataset
Risk-engine host	Ethernet, HTTP on localhost	Runs the model, the API, SQLite and the Ethereum client	Windows 11 laptop, Python 3.10
Ethereum node	JSON-RPC over HTTP, port 8545	Hosts the audit contract	Ganache 7.9.2 (local, chain id 1337)

The SIM800L is controlled with the AT command set [21]. The gateway uses five commands, listed with the software interfaces in Section 3.3. The microcontroller never sends raw modem output to the engine: it extracts the fields it needs and signs the resulting packet, so the engine verifies an authenticated structure and not text that could be edited in transit.

3.3 SOFTWARE INTERFACES

The software stack and its versions in the measured prototype are listed below. All components are open source.

Table 3.2 – Software stack

Component	Version	Use
Python	3.10.11	Model, risk engine, API, experiments, report tooling
XGBoost	3.2.0	Gradient-boosted classifier and native TreeSHAP
scikit-learn / pandas / NumPy	1.7.2 / 2.3.3 / 2.2.6	Baselines, metrics, data handling
FastAPI / Uvicorn / Pydantic	0.140.13 / 0.51.0 / 2.13.4	REST service, validation, OpenAPI documentation
web3.py	8.0.0	Ethereum JSON-RPC client
Solidity compiler (solc-js)	0.8.24	Compiles SIMSwapAuditLog.sol (optimiser on, EVM version shanghai)
Ganache	7.9.2	Local Ethereum node
Node.js	24.x	Runs the compiler script and Ganache
SQLite	3.40.1	Off-chain decision store
pytest	9.1.1	43 automated tests
Playwright + Chrome	1.62.0	Console screenshots and a JavaScript-error smoke test
Browser	HTML, CSS, JavaScript (no framework)	Operations console

REST API. The risk engine exposes the following endpoints under /api/v1. Request and response bodies are JSON, validated by Pydantic models; invalid input returns 422, doctored telemetry 401 and a repeated request identifier 409.

Table 3.3 – REST API endpoints

Method and path	Purpose
POST /evaluate	Evaluate one SIM replacement request; returns the decision, reasons, anchor and timings
GET /decisions, /decisions/{id}	List stored decisions (filter by tier) or fetch one
POST /decisions/{id}/resolve	Analyst outcome for a MANUAL_REVIEW decision, recorded on-chain
GET /ledger	On-chain entries, counters per tier and contract information
GET /audit/verify	Recompute every record's hashes and compare with the chain
POST /demo/tamper, /demo/restore	Simulate and undo an insider edit of the off-chain store
POST /demo/edge-attack/{kind}	Submit doctored telemetry (altered, replayed, stale, unknown device)
GET /scenarios, /scenarios/{key}	Real held-out requests, freshly signed, for the console presets
GET /metrics, /health, /edge/rejections	Model comparison and policy; service status; recent rejected packets

Signed telemetry packet. The structure below is what the edge gateway sends and the engine verifies. The signature is an HMAC-SHA256 over the canonical JSON of the other four fields (sorted keys, no whitespace, integral numbers written as integers):

```
{
  "device_id": "EDGE-STORE-CHN-01",
  "nonce": "3e49e9406bca4963bb260f35d58d526b",
  "ts": 1791430483.471,
  "payload": {
    "imei": "352092114951370", "rssi_dbm": -71, "cell_id": 51890,
"roaming_flag": 0,
    "imei_changed": 1, "device_age_days": 0, "old_sim_last_seen_min": 12.4,
    "distance_from_home_km": 40.0, "cell_changes_24h": 3
  },
  "signature": "7b8a1d1ef6c1...d4fbf7"
}
```

Modem AT commands. The gateway issues the following commands to the SIM800L:

Table 3.4 – AT commands used by the edge gateway

Command	Returns	Used for
AT+GSN	15-digit IMEI of the handset	Handset identity; blacklist lookup; Luhn check
AT+CCID	SIM serial number	Identity of the SIM in the slot
AT+CSQ	Signal quality 0-31	RSSI in dBm = -113 + 2 x value
AT+CREG?	Registration status (1 home, 5 roaming)	roaming_flag
AT+CENG?	Serving cell identifier	Cell id; cell-change count

Smart contract events. Applications that monitor the ledger subscribe to three events: DecisionLogged (entry id, subscriber reference, tier, risk in basis points, decision hash, SHAP hash), ReviewResolved (entry id, final tier, analyst reference, note hash) and EngineAuthorized / EngineRevoked. The contract interface is summarised in Table 2.9 and the full source is in the Sample Coding section.

CHAPTER 4

TESTING

4.1 TEST PLAN

The test plan verifies that every module behaves as specified, that the modules work together, and that the complete system makes accurate, explainable and tamper-evident decisions under normal and hostile conditions. Testing is conducted at four levels: unit tests of individual modules, integration tests across module boundaries, system tests of the complete decision pipeline against a real blockchain, and security and resilience tests that attack the telemetry and the off-chain store. Two kinds of evidence are produced: automated tests, which give a pass or fail result and are repeated on every change, and experiments, which measure quantities such as detection rate, latency and gas on the held-out data.

4.1.1 Unit Testing

Each module was tested in isolation. The rule engine was tested at the boundary of every rule (for example 11 versus 12 swaps by one agent) and for the tier each severity forces. The telemetry verifier was tested against a genuine packet and against altered, replayed, stale, mis-keyed, unknown-device, bad-checksum and malformed packets. The hashing functions were tested for determinism, independence of key order, and sensitivity to a change in every covered field. The explanation module was tested for additivity, namely that the base value plus the SHAP contributions equals the model's output, and for correct ordering of the reasons. The smart contract was tested on a real chain for access control, input validation, the resolution rules and the absence of any function that edits or deletes an entry.

4.1.2 Integration Testing

Integration tests exercise the API with the real model, database and blockchain together. They confirm that a signed packet produced by the gateway

emulator is accepted, that the decision returned matches the tier expected for each scenario, that the entry written to the chain verifies against the stored record, that only a MANUAL_REVIEW decision can be resolved and only once, and that a new database does not inherit another database's audit trail. Integration testing also covered the browser: a scripted Chrome session drives every view of the console and fails on any JavaScript error.

4.1.3 System Testing

System tests pass 300 randomly selected held-out requests through the complete engine on a fresh contract, recording each decision, its timing and its gas. The tier returned by the online path is compared with the tier computed offline by the vectorised evaluation, which must agree for every request. The audit is then run over all anchored decisions.

4.1.4 Security and Resilience Testing

Three groups of hostile scenarios were executed. **Off-chain tampering:** an insider changes a decision, lowers a risk score, alters a SHAP contribution or an input feature, clears the rule flags, relabels the model version, deletes a record or inserts a forged one. **Telemetry attacks:** a field is altered in transit, a valid packet is replayed, a packet is submitted an hour late, a packet comes from an unregistered device, a correctly signed packet carries an impossible IMEI, and a packet is signed with the wrong key. **Baseline comparison:** a local hash chain that an insider can recompute is attacked in the same way to show what the on-chain anchor adds.

Table 4.1 – Test levels

Level	Scope	Tooling	Automated tests
Unit	Rules, telemetry, hashing and SHAP, smart contract	pytest, local Ganache chain	32
Integration / system	API with model, SQLite and chain; resolution; audit; telemetry attacks	pytest, FastAPI test client	11
Browser smoke test	All console views, JavaScript errors	Playwright, Chrome	script
Experiments	Accuracy, policy, latency, gas, tamper detection, edge attacks	experiments/run_experiments.py	script

4.2 TEST PROCEDURE

The test environment was a Windows 11 laptop running Python 3.10, Node.js 24 and a local Ganache blockchain (chain id 1337). All services ran on one machine, so timings include no network latency between the engine, database and chain. The dataset (20,000 requests, 1,000 fraudulent) was generated with fixed seeds and split once into training (12,000; 600 fraud), validation (4,000; 200 fraud) and test (4,000; 200 fraud) sets, stratified by class.

The procedure follows four rules that keep the reported numbers honest. **First,** the test set is used only to report; the operating threshold of every model, and the review and block thresholds of the risk engine, are chosen on the validation set. **Second,** every model, including the baselines, is trained on the same training set and evaluated on the same test set. **Third,** the online engine and the offline evaluation use the same rule and policy code, and the experiment fails if they disagree. **Fourth,** every number in this chapter is written to a results file by a script and inserted into this report from that file.

The whole pipeline is reproduced with three commands: the first starts the local Ethereum node, the second generates the data, trains and calibrates the model, compiles the contract, runs the experiments and the test suite, and the third starts the console:

```
python run_pipeline.py --chain      # terminal 1: local Ethereum node
(Ganache)
python run_pipeline.py --all        # terminal 2: data, model, contract,
experiments, tests
python run_pipeline.py --serve      # operations console on
http://127.0.0.1:8051
```

4.3 TEST RESULTS

The results below are on the held-out test set unless stated otherwise. They describe the prototype on synthetic data and must not be read as the performance an operator would see on real traffic; the limitations are set out at the end of this chapter.

Model performance

Table 4.2 – Model comparison on the held-out test set

Model	Precision	Recall	F1	ROC-AUC	PR-AUC	Threshold
XGBoost (proposed)	**0.812**	**0.780**	**0.796**	**0.948**	**0.854**	**0.27**
Random Forest	0.827	0.740	0.781	0.944	0.836	0.46
Logistic Regression	0.504	0.645	0.566	0.907	0.631	0.80
Isolation Forest	0.155	0.520	0.239	0.795	0.154	0.49
Static rule checklist	0.491	0.395	0.438	0.750	0.285	2.00

XGBoost achieves the best F1 (0.796), ROC-AUC (0.948) and PR-AUC (0.854). The random forest is close (PR-AUC 0.836), and with only 200 fraudulent test requests a single split cannot separate the two with confidence; five-fold cross-validation of XGBoost gives ROC-AUC 0.958 +/- 0.013 and PR-AUC 0.852 +/- 0.020 across folds. Logistic regression (0.907) shows that non-linear interactions matter. The unsupervised isolation forest (0.795) performs poorly because much legitimate behaviour, such as travel and eSIM upgrades, is also unusual. The static checklist (0.750) is the weakest. Fig. 4.1 gives the ROC and precision-recall curves and Fig. 4.2 the confusion matrix of XGBoost at its validation-chosen threshold: 156 of 200 fraudulent requests are detected and 36 of 3,800 legitimate requests are flagged. Scoring one request takes 2.6 ms.

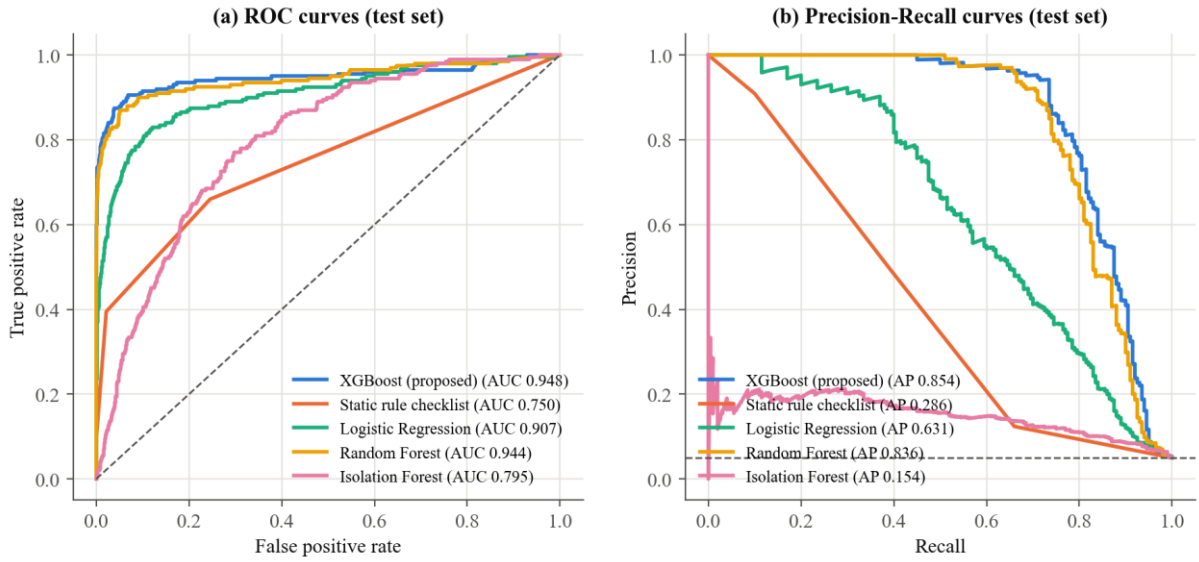


Fig. 4.1 – ROC and precision-recall curves of the five models (test set)

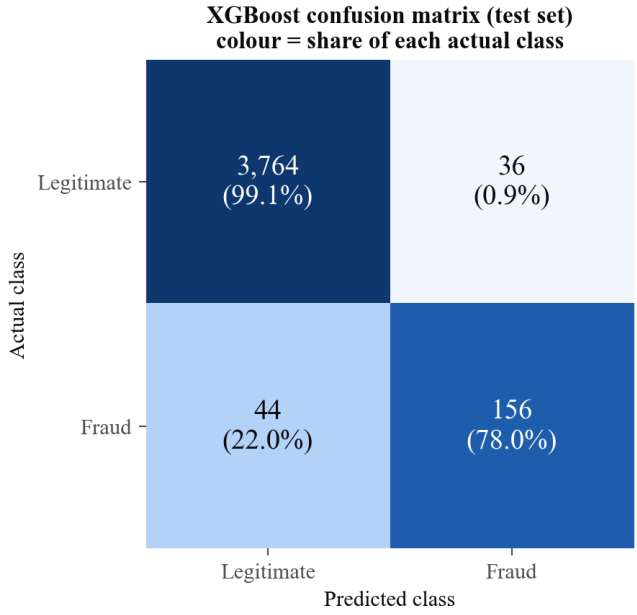


Fig. 4.2 – Confusion matrix of XGBoost (test set)

Explanation quality

The TreeSHAP contributions are additive to within 5.7e-06 log-odds. Fig. 4.3 ranks the features by mean absolute SHAP value: distance from home cell (0.82), handset age on network (0.59), authentication failures (0.51), time since the old SIM was seen (0.42), handset change (0.39), agent's swaps in 24 h (0.33). Weekend, prior swaps and roaming contribute almost nothing (0.008 to 0.021). This ordering is what the attack model predicts, so it confirms that the model learned the intended structure; because the data are synthetic it is not evidence

about real traffic. Fig. 4.4 shows the direction of each effect: long distances, young handsets, many authentication failures and an active old SIM push the score towards fraud.

Fig. 4.5 explains one detected request (REQ-011096, probability 0.93). Starting from the base value of -2.97, a request made 50.7 km from home adds 2.75, a recent profile change 1.38 and a missing biometric check 0.89, while the old SIM having been quiet for 112 minutes subtracts 0.44. This is the explanation an analyst reads in the console and the explanation whose hash is anchored on-chain.

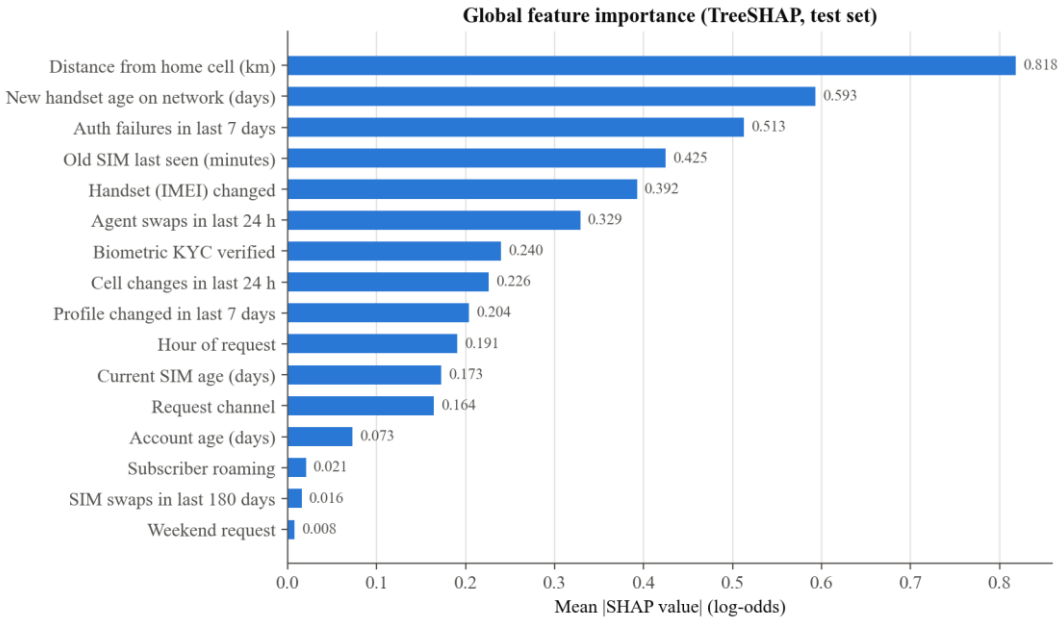


Fig. 4.3 – Global feature importance (mean |SHAP|, test set)

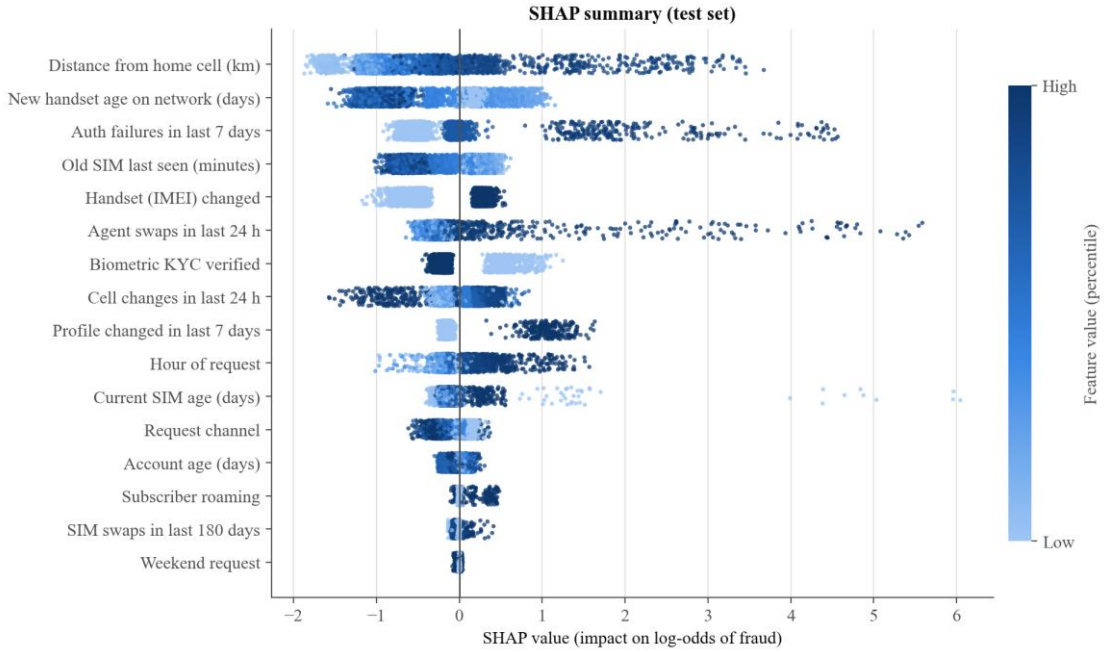


Fig. 4.4 – SHAP summary: direction and size of each feature's effect

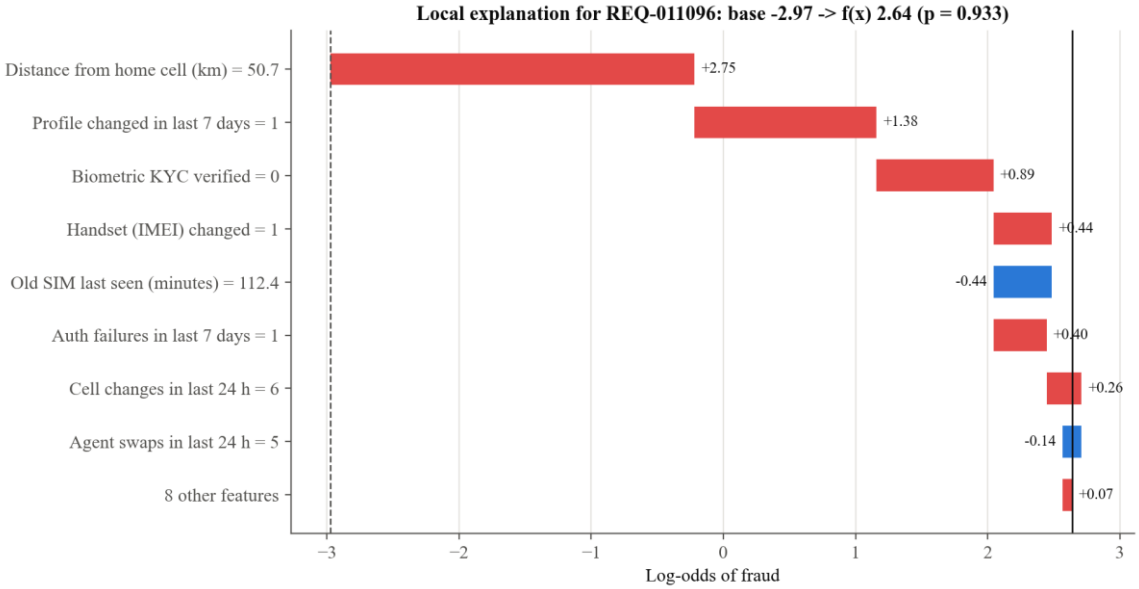


Fig. 4.5 – Local explanation of one fraudulent request

Policy performance and the value of each layer

Table 4.3 – Three-tier policy and ablations on the test set

Policy	Fraud caught %	Fraud blocked %	Reviewed % of all	Legit blocked	Fraud missed	Flag precision %	Cost
Static checklist (today)	39.5	10.0	3.5	2	121	49.1	12,279
Rules only	36.0	10.0	3.7	7	128	41.1	13,088
ML only (XGBoost)	82.0	43.5	4.0	0	36	66.9	3,758
Hybrid risk engine (proposed)	**84.5**	**54.0**	**6.8**	**7**	**31**	**43.8**	**3,511**

Fraud caught means sent to manual review or blocked; cost uses the calibration weights (100 per missed fraud, 20 per false block, 1 per review). Four findings stand out (Fig. 4.6, Fig. 4.7).

1. **The static checklist is far behind.** It catches 39.5% of fraud and misses 121 of 200 fraudulent requests; rules alone do no better (36.0%).
2. **The model supplies most of the detection.** XGBoost alone catches 82.0% of fraud, blocks 43.5% of it automatically and blocks no legitimate customer.
3. **The hybrid engine adds a modest gain at a visible cost.** It catches 84.5% and blocks 54.0% of fraud automatically, but sends 6.8% of all requests to analysts (against 4.0%), lowers the share of flagged requests that are really fraud from 66.9% to 43.8%, and blocks 7 legitimate customers. All 7 are caused by the blacklist rule: second-hand handsets that were reported stolen after sale.
4. **Whether the hybrid engine is worth it depends on the cost of a false block.** Under the calibration weights it costs 3,511 against 3,758 for ML only, a 6.6% saving, but the advantage disappears if a false block costs more than about 55 units. The weights are assumptions that an operator should replace with its own.

The rules are justified less by their marginal accuracy than by what they guarantee. A blacklisted handset is always blocked whatever the model says, and a swap within seven days of the last one is always reviewed; these are policy commitments that a probabilistic model cannot give.

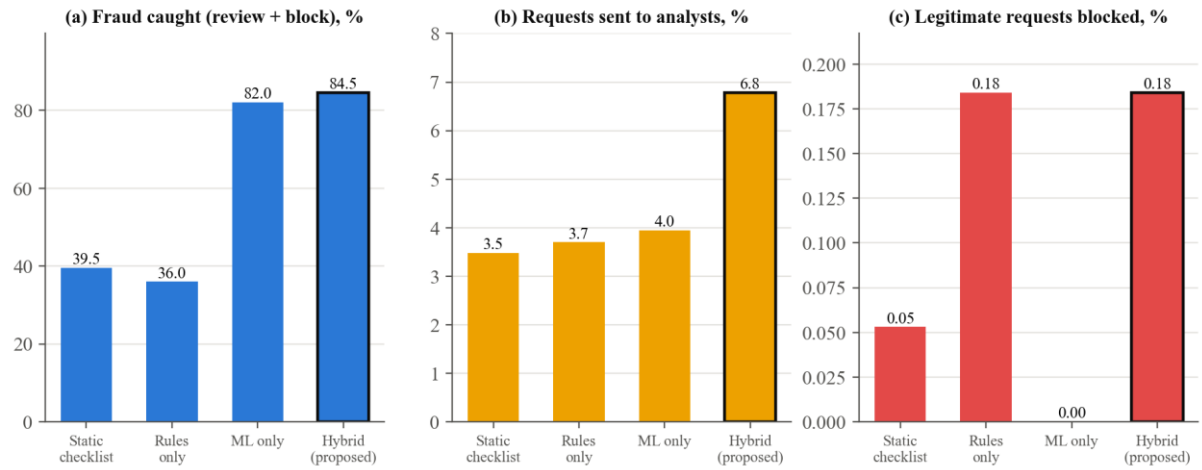


Fig. 4.6 – Fraud caught, analyst workload and false blocks for the four policies

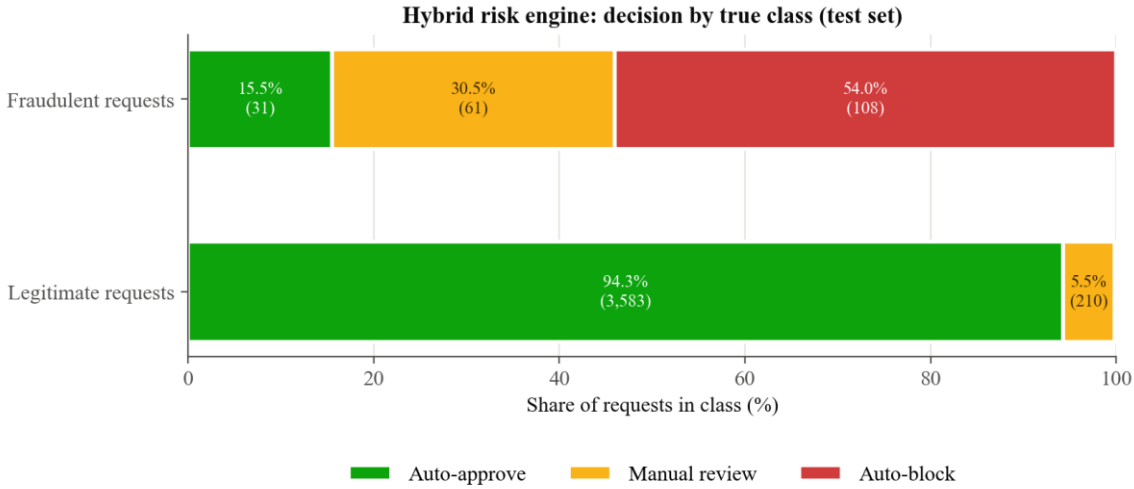


Fig. 4.7 – Decision by true class under the hybrid risk engine

Detection by attack type

Fig. 4.8 breaks the hybrid engine's result down by the attack archetype that generated each fraudulent test request. Social engineering (94.4% caught, 69.7% blocked), insider retailer fraud (95.7%) and rapid re-swaps (92.3%) are detected well. Forged-identity swaps in stores are the weakest real attack (75.7% caught, only 27% blocked), because they are made in person, usually pass biometric checks and are often near the victim's home. **Stealth fraud, drawn from the legitimate distribution, is essentially undetected** (6.7%, one request in 15); no model can find an attack that leaves no trace in its inputs, and the design therefore relies on the authenticated radio-side signals that real attacks do disturb.

Compared with the model alone, the rules add up to 3.4 points for an archetype and nothing for rapid re-swaps.

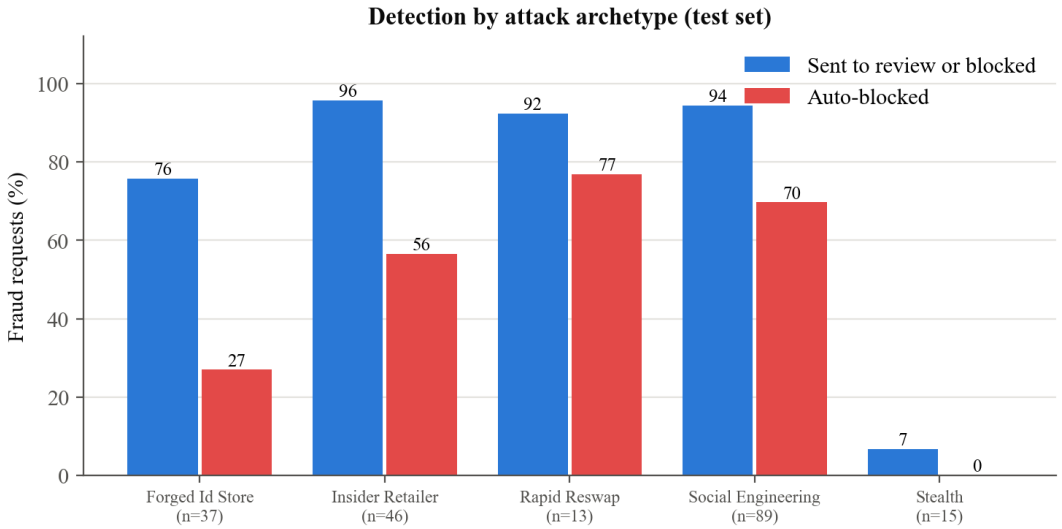


Fig. 4.8 – Detection by attack archetype (test set)

Review-threshold trade-off

Fig. 4.9 shows how the review threshold trades analyst workload against fraud caught. Lowering the threshold to 0.02 would catch 93.5% of fraud but send 25% of all requests to analysts, far above the 8% capacity assumed in the calibration. The selected threshold of 0.13 sits at the knee of the curve: 6.8% reviewed for 84.5% caught.

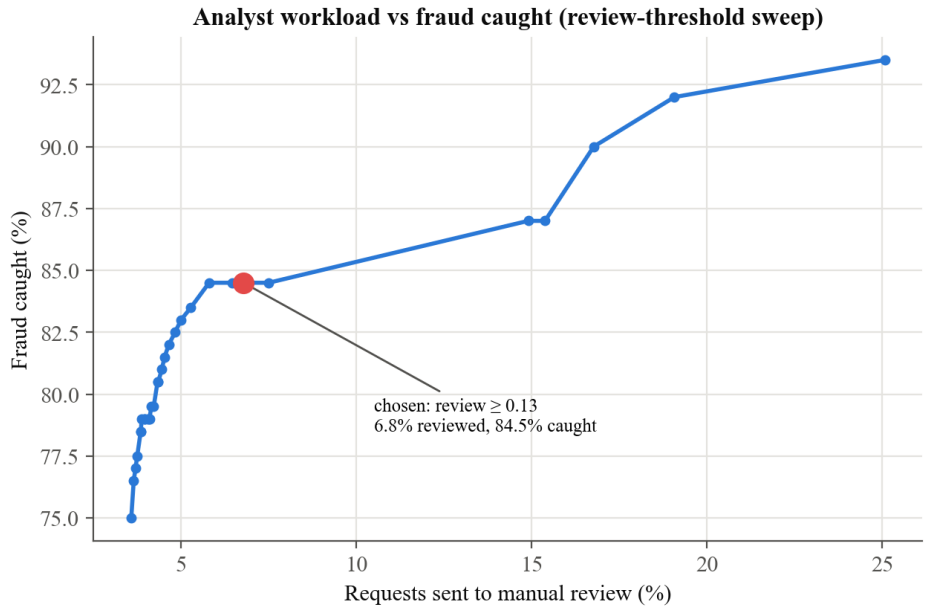


Fig. 4.9 – Analyst workload against fraud caught as the review threshold varies

Latency and throughput

Table 4.4 – Processing time per decision (n = 300, local chain)

Step	Median (ms)	95th percentile (ms)	Mean (ms)
Telemetry verification	2.6	9.4	3.8
Model scoring and SHAP	2.0	5.8	2.8
Rules and blacklist lookup	1.5	4.5	2.0
Canonical hashing	0.3	0.9	0.4
Blockchain transaction	117.4	349.9	168.4
Database write	13.9	35.1	18.6
Total	**143.5**	**393.4**	**197.4**

Excluding the blockchain write, a decision takes 21 ms at the median and 59 ms at the 95th percentile. Including it, the median is 143 ms and the 95th percentile 393 ms, and the sequential throughput is 4.8 decisions per second (Fig. 4.10). The chain transaction accounts for 82% of the median. Timings depend on the machine and its load: two complete runs on the same laptop gave median totals of 115 ms and 143 ms. On a public network, block times of seconds would dominate further, so a production deployment should return the decision first and anchor it through a queue.

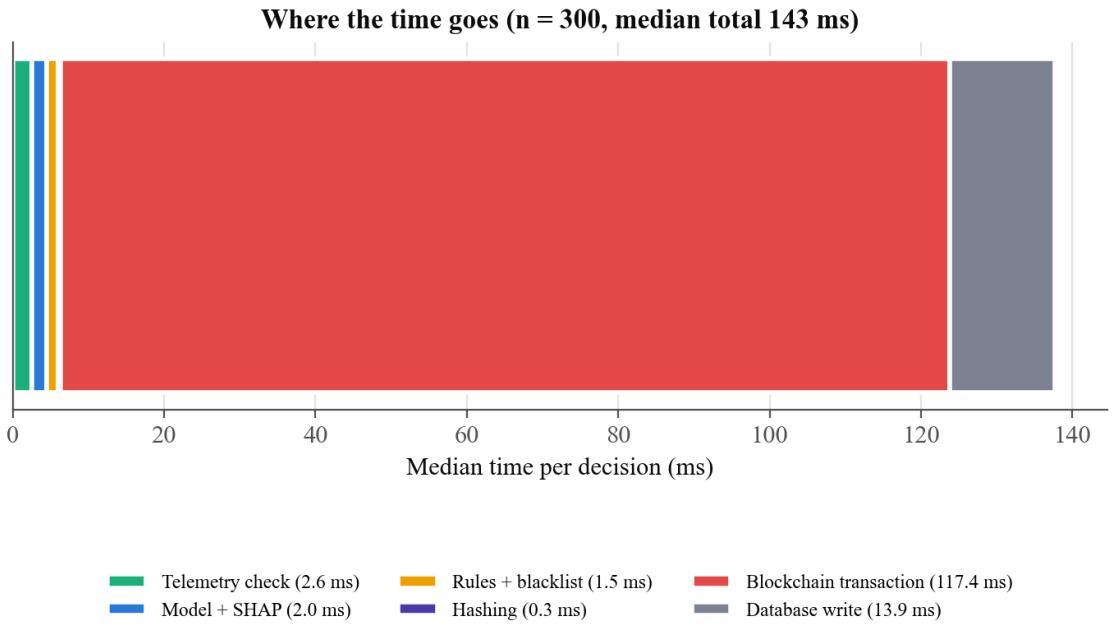


Fig. 4.10 – Where the time goes in one decision

Gas and storage cost

Table 4.5 – Gas measured on the local chain

Operation	Gas used
Contract deployment (once)	909,098 (3,732 bytes of bytecode)
logDecision: median	172,240
logDecision: minimum / maximum	172,204 / 189,364
resolveReview: median	142,728
logDecision at an assumed 20 gwei (mean gas)	0.0034 ETH

An anchored decision costs about 172,240 gas: four storage slots for the entry, an index entry per subscriber, one counter update and the event. Gas is measured exactly by the EVM, but the price in ether is an assumption, since gas prices differ by orders of magnitude between public networks, layer-2 networks and permissioned chains. Anchoring a Merkle root of many decisions in one transaction, listed in Chapter 5, would reduce the cost per decision substantially.

Tamper detection

Table 4.6 – Detection of tampering with the off-chain store

Insider action	Trials	Detected	Rate
Change the decision	100	100	100%
Lower the risk score	100	100	100%
Alter the SHAP explanation	100	100	100%
Alter an input feature value	100	100	100%
Clear the rule flags	47	47	100%
Relabel the model version	100	100	100%
Delete a record	15	15	100%
Insert a forged record	15	15	100%
Baseline: edit a local hash chain, insider recomputes the hashes	100	0	0%

Every edit, deletion and forged insertion was detected by the audit, and the audit passed again after the records were restored. The baseline shows why the chain matters: a hash chain stored with the records is rewritten consistently by an insider who has write access, and the verification then passes, so it detected none of the edits. Verifying 300 anchored decisions took 8.7 s (29 ms per entry, dominated by one chain read per entry). The audit cannot detect a decision that was never stored anywhere; the operational control for that case is that the response returned to the agent carries the ledger entry, which the CRM must record before the swap is executed.

Telemetry integrity

Table 4.7 – Edge telemetry attacks

Packet	Trials	Outcome
Genuine telemetry (must be accepted)	100	100 accepted
Payload field altered	100	100 rejected
Packet replayed	100	100 rejected
Stale packet (1 h old)	100	100 rejected
Unregistered device	100	100 rejected
Invalid IMEI checksum	100	100 rejected
Wrong signing key	100	100 rejected

All doctored packets were rejected and no genuine packet was. Because the checks are deterministic cryptography, the value of this test lies less in the rate than in the integration defects it exposed. While testing the console, valid packets were being rejected because the browser's JSON round trip rewrote 12.0 as 12, changing the bytes that had been signed; the canonical encoding now writes integral numbers as integers on both sides, and a regression test covers it. A unit test also found that a signed envelope kept a reference to the caller's payload, so editing the envelope silently edited the original; the envelope now takes a copy.

Automated test suite

Table 4.8 – Automated tests

Test module	Tests	Covers
test_rules	8	Every rule boundary; tier forced by each severity; score cap
test_telemetry	10	Genuine, altered, replayed, stale, unknown, mis-keyed, bad-checksum, malformed packets; JavaScript round trip
test_hashing_and_explain	5	Hash determinism and sensitivity; pseudonym; SHAP additivity; reason ordering
test_contract	9	Read-back; append-only ABI; access control; revocation; score bound; resolution rules; subscriber index
test_api	11	Scenarios per tier; anchor and audit; privacy; 401, 409, 422; insider edits; resolution; fresh-database isolation
Total	**43**	**All pass**

Discussion and limitations

The results support the design claims, but only within these limits.

- **Synthetic data.** Absolute figures describe the generator, not an operator's traffic. The dataset was built so that fraud is detectable but not trivially separable, and the share of stealth fraud and hard legitimate cases was chosen by us. The comparisons between methods are more informative than the absolute values.
- **Small number of positives.** With 200 fraudulent test requests, the 95% interval on XGBoost's recall of 0.78 is about +/- 6 points; differences of a few points between close methods are within noise.

- **Emulated edge and local chain.** No radio measurement was made, and the chain is a single-node test network, so confirmation times and gas prices do not describe a public network. Gas units are exact.
- **Assumed costs and capacity.** The policy thresholds depend on the stated cost weights and an 8% review capacity.
- **Non-adaptive attacker.** The attacks are fixed by the generator; an attacker who learns the rules and shapes requests to stay below them was not simulated.
- **Integrity is not truth.** Anchoring proves what was decided and that records were not changed afterwards. It cannot prove that the inputs were accurate; signed telemetry narrows that gap but a compromised gateway can still sign false readings.

CHAPTER 5

FUTURE ENHANCEMENTS

Although SIMGuard provides a working and measured solution for explainable, tamper-evident SIM replacement decisions, several enhancements would move it from a prototype towards an operator deployment.

Asynchronous Anchoring with Merkle Batching

Return the decision to the agent immediately and anchor it through a queue. Decisions would be collected for a short interval and a single Merkle root of their hashes written in one transaction, with an inclusion proof stored beside each record. This removes the blockchain from the decision's critical path, which Section 4.3 shows dominates latency, and reduces the gas cost per decision to a small fraction of the current figure.

Public Testnet and Consortium Deployment

Deploy the contract to a public test network such as Sepolia, then to a permissioned or consortium chain in which several operators, and the regulator, run nodes. A multi-signature owner and several authorised engine keys would remove the single signing account of the prototype and make an operator's own insiders unable to stop anchoring unnoticed.

Pilot on Real Operator Data

Replace the synthetic dataset with anonymised historical SIM replacement records that carry confirmed fraud labels, using a time-based split and the operator's own cost of a missed fraud, a false block and a review. This is the only way to establish real detection performance and to recalibrate the thresholds, and it should be accompanied by drift monitoring on the feature distributions.

Physical Edge Gateway with a Secure Element

Build the SIM800L and microcontroller gateway described in Chapter 3, measure real signal and cell observations, and move the device key into a hardware secure element so that it cannot be extracted from a stolen gateway.

Remote attestation would let the engine check the gateway's firmware as well as its packets.

Behavioural and Graph Features

Add signals observed after the swap, such as sudden changes in data usage or the first login from a new device, and graph features that link agents, handsets, IMEIs and accounts, so that rings of related requests are visible even when each request looks ordinary. Sequence models could use the customer's history directly.

Analyst Feedback Loop

Use analysts' resolutions, already recorded as separate on-chain entries, as labels for periodic retraining, and use active learning to route the most informative borderline requests to analysts first.

Adversarial Robustness

Simulate an attacker who learns the rules and the model and shapes requests to stay below the thresholds, then harden the model with adversarial training and monotonic constraints on signals that should only ever increase risk.

Privacy-Preserving Verification

Allow a regulator to confirm that a decision followed the stated policy without seeing the explanation, using selective disclosure or zero-knowledge proofs, and rotate the subscriber pseudonym key periodically to limit long-term linkage.

Coverage of eSIM Provisioning and Number Porting

Extend the same decision, explanation and anchoring path to eSIM profile downloads and port-out requests, which are exploited in the same way as physical SIM replacement.

CHAPTER 6
CONCLUSION

This project has designed, implemented and evaluated SIMGuard, an explainable and tamper-evident decision system for SIM replacement requests. It makes three contributions: an authenticated edge-telemetry path that lets the engine trust the radio-side signals an attacker cannot talk its way around; a hybrid risk engine that combines an XGBoost classifier, exact TreeSHAP explanations, deterministic rules and an IMEI blacklist into a calibrated three-tier decision; and a Solidity audit contract that anchors fingerprints of both the decision and its explanation while keeping personal data off the chain.

On a held-out set of 4,000 synthetic requests, XGBoost reached ROC-AUC 0.948 against 0.750 for a static checklist, and the risk engine sent 84.5% of fraudulent requests to review or block against 39.5%, with 6.8% of requests reviewed by analysts. The audit detected 577 of 577 tampering attempts on the off-chain store, whereas a hash chain that an insider can recompute detected none, and every doctored telemetry packet was rejected. A decision is returned in a median of 21 ms and anchored for about 172,240 gas. Forty-three automated tests pass.

The results also mark the limits of the approach. The data are synthetic, so the accuracy figures describe the prototype and not an operator's traffic; fraud that leaves no trace in the inputs cannot be detected; the rules improve recall only modestly and cost extra reviews and a few false blocks, so their value is mainly the policy guarantees they give; and anchoring proves what was decided, not that the inputs were true. These limits shape the enhancements proposed in Chapter 5, chiefly a pilot on real data, a physical edge gateway and asynchronous, batched anchoring.

In conclusion, the project shows that a SIM replacement decision can be made accurate, explained in terms an analyst can act on, and recorded so that neither the decision nor its reasoning can be quietly rewritten, using commodity hardware and open software. Treating the explanation as part of the evidence, and anchoring it alongside the decision, is the step that turns an opaque fraud score into an accountable one.

REFERENCES

[1] 3GPP, "Numbering, addressing and identification," 3rd Generation Partnership Project, Technical Specification TS 23.003.

[2] Bhushan et al., "SIM Swap Fraud Detection Using AI ML," in Proc. FICTA 2025, Springer, 2026. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-032-20121-8_27

[3] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

[4] V. Buterin, "A Next-Generation Smart Contract and Decentralized Application Platform" (Ethereum White Paper), 2014.

[5] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785-794.

[6] F. Doshi-Velez and B. Kim, "Towards A Rigorous Science of Interpretable Machine Learning," arXiv:1702.08608, 2017.

[7] European Parliament and Council, "Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data (General Data Protection Regulation)," Official Journal of the European Union, L 119, 2016.

[8] Federal Bureau of Investigation, Internet Crime Complaint Center, "Criminals Increasing SIM Swap Schemes to Steal Millions of Dollars from US Public," Public Service Announcement I-020822-PSA, Feb. 8, 2022. [Online]. Available: https://www.ic3.gov/PSA/2022/PSA220208

[9] Government of India, "The Digital Personal Data Protection Act, 2023," Act No. 22 of 2023.

[10] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," IETF RFC 2104, Feb. 1997.

[11] K. Lee, B. Kaiser, J. Mayer, and A. Narayanan, "An Empirical Study of Wireless Carrier Authentication for SIM Swaps," in Proc. 16th Symp. Usable Privacy and Security (SOUPS), 2020, pp. 61-79.

[12] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in Proc. 8th IEEE Int. Conf. Data Mining (ICDM), 2008, pp. 413-422.

[13] S. M. Lundberg et al., "From local explanations to global understanding with explainable AI for trees," Nature Machine Intelligence, vol. 2, no. 1, pp. 56-67, 2020.

[14] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 4765-4774.

[15] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[16] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, Aug. 2015.

[17] S. Ramirez, "FastAPI," web framework documentation. [Online]. Available: https://fastapi.tiangolo.com

[18] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," PLOS ONE, vol. 10, no. 3, e0118432, 2015.

[19] "A Machine Learning and Blockchain Based Efficient Fraud Detection Mechanism," Sensors, vol. 22, no. 19, art. 7162, 2022. [Online]. Available: https://www.mdpi.com/1424-8220/22/19/7162

[20] L. S. Shapley, "A value for n-person games," in Contributions to the Theory of Games, vol. II, H. W. Kuhn and A. W. Tucker, Eds. Princeton, NJ, USA: Princeton Univ. Press, 1953, pp. 307-317.

[21] SIMCom Wireless Solutions, "SIM800 Series AT Command Manual."

[22] Ethereum Foundation, "Solidity Documentation," version 0.8.24. [Online]. Available: https://docs.soliditylang.org

[23] I. T. Sotonye, N. Eyidia, and W. N. Abidde, "Combating SIM Swap Fraud in Telecommunications: A Machine Learning Approach and Multi-Factor Authentication as a Preventive Strategy," Journal of Advancement in Communication System, vol. 8, no. 2, pp. 22-27, 2025, doi: 10.5281/zenodo.15591954.

[24] Truffle Suite, "Ganache: a personal blockchain for Ethereum development," version 7.9.2.

[25] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Ethereum Project Yellow Paper, 2014.

SAMPLE CODING

Audit Smart Contract (Solidity) - SIMSwapAuditLog.sol

The contract appends one entry per decision and one optional resolution per manual review. It has no function that edits or deletes an entry; only addresses authorised by the owner can write; tier counters are updated on write so statistics are read in constant time.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

/**
 * @title  SIMSwapAuditLog
 * @notice Append-only, tamper-evident audit trail for SIM replacement
decisions.
 *
 * What goes on-chain
 *   - a pseudonymous subscriber reference (HMAC of the MSISDN, never the
number)
 *   - the three-tier decision and the composite risk score (basis points)
 *   - SHA-256 of the canonical decision record
 *   - SHA-256 of the canonical SHAP explanation (the reasoning behind the
decision)
 * The full record and the human-readable explanation stay off-chain in the
 * operator's database. Anyone can later recompute both hashes from that
record and
 * compare them with the values stored here, so any after-the-fact edit is
exposed.
 *
 * Access control
 *   - owner: authorises / revokes risk-engine addresses
 *   - authorised engine: the only address allowed to append entries
 *   - nobody (including the owner) can modify or delete an existing entry
 */
contract SIMSwapAuditLog {
    enum Decision { AUTO_APPROVE, MANUAL_REVIEW, AUTO_BLOCK }

    // Packs into four storage slots.
    struct AuditEntry {
        bytes32 subscriberRef;
        bytes32 decisionHash;
        bytes32 shapHash;
        uint48  timestamp;
        uint16  riskBps;       // composite risk score x 10000
        Decision decision;
        address loggedBy;
    }

    struct Resolution {
        Decision finalDecision;
        bytes32  analystRef;   // pseudonymous analyst identifier
        bytes32  noteHash;     // SHA-256 of the analyst's written
justification
        uint48   timestamp;
        address  resolvedBy;
```

```
    }

    address public owner;
    uint256 public entryCount;
    uint256[3] public decisionCounts;

    mapping(uint256 => AuditEntry) private entries;
    mapping(uint256 => Resolution) private resolutions;
    mapping(uint256 => bool) public isResolved;
    mapping(address => bool) public authorizedEngines;
    mapping(bytes32 => uint256[]) private subscriberEntries;

    event DecisionLogged(
        uint256 indexed entryId,
        bytes32 indexed subscriberRef,
        Decision decision,
        uint16 riskBps,
        bytes32 decisionHash,
        bytes32 shapHash
    );
    event ReviewResolved(
        uint256 indexed entryId,
        Decision finalDecision,
        bytes32 analystRef,
        bytes32 noteHash
    );
    event EngineAuthorized(address indexed engine);
    event EngineRevoked(address indexed engine);
    event OwnershipTransferred(address indexed previousOwner, address
indexed newOwner);

    error NotOwner();
    error NotAuthorizedEngine();
    error InvalidScore();
    error UnknownEntry();
    error NotPendingReview();
    error AlreadyResolved();
    error InvalidFinalDecision();
    error ZeroAddress();

    modifier onlyOwner() {
        if (msg.sender != owner) revert NotOwner();
        _;
    }

    modifier onlyEngine() {
        if (!authorizedEngines[msg.sender]) revert NotAuthorizedEngine();
        _;
    }

    constructor() {
        owner = msg.sender;
        authorizedEngines[msg.sender] = true;
        emit OwnershipTransferred(address(0), msg.sender);
        emit EngineAuthorized(msg.sender);
    }

    // ------------------------------------------------------------------
admin

    function authorizeEngine(address engine) external onlyOwner {
```

```
        if (engine == address(0)) revert ZeroAddress();
        authorizedEngines[engine] = true;
        emit EngineAuthorized(engine);
    }

    function revokeEngine(address engine) external onlyOwner {
        authorizedEngines[engine] = false;
        emit EngineRevoked(engine);
    }

    function transferOwnership(address newOwner) external onlyOwner {
        if (newOwner == address(0)) revert ZeroAddress();
        emit OwnershipTransferred(owner, newOwner);
        owner = newOwner;
    }

    // ------------------------------------------------------------ write
path

    function logDecision(
        bytes32 subscriberRef,
        Decision decision,
        uint16 riskBps,
        bytes32 decisionHash,
        bytes32 shapHash
    ) external onlyEngine returns (uint256 id) {
        if (riskBps > 10000) revert InvalidScore();

        id = entryCount++;
        entries[id] = AuditEntry({
            subscriberRef: subscriberRef,
            decisionHash: decisionHash,
            shapHash: shapHash,
            timestamp: uint48(block.timestamp),
            riskBps: riskBps,
            decision: decision,
            loggedBy: msg.sender
        });
        subscriberEntries[subscriberRef].push(id);
        decisionCounts[uint8(decision)]++;

        emit DecisionLogged(id, subscriberRef, decision, riskBps,
decisionHash, shapHash);
    }

    /// @notice Record the human analyst's outcome for a MANUAL_REVIEW
entry.
    ///         The original entry is never modified; the resolution is a
second record.
    function resolveReview(
        uint256 entryId,
        Decision finalDecision,
        bytes32 analystRef,
        bytes32 noteHash
    ) external onlyEngine {
        if (entryId >= entryCount) revert UnknownEntry();
        if (entries[entryId].decision != Decision.MANUAL_REVIEW) revert
NotPendingReview();
        if (isResolved[entryId]) revert AlreadyResolved();
        if (finalDecision == Decision.MANUAL_REVIEW) revert
InvalidFinalDecision();
```

```
        isResolved[entryId] = true;
        resolutions[entryId] = Resolution({
            finalDecision: finalDecision,
            analystRef: analystRef,
            noteHash: noteHash,
            timestamp: uint48(block.timestamp),
            resolvedBy: msg.sender
        });
        emit ReviewResolved(entryId, finalDecision, analystRef, noteHash);
    }

    // ------------------------------------------------------------ read
path

    function getEntry(uint256 id) external view returns (AuditEntry memory)
{
        if (id >= entryCount) revert UnknownEntry();
        return entries[id];
    }

    function getResolution(uint256 id) external view returns (Resolution
memory) {
        if (!isResolved[id]) revert UnknownEntry();
        return resolutions[id];
    }

    /// @notice Compare off-chain recomputed hashes with the anchored ones.
    function verifyEntry(uint256 id, bytes32 decisionHash, bytes32 shapHash)
        external
        view
        returns (bool decisionOk, bool shapOk)
    {
        if (id >= entryCount) revert UnknownEntry();
        AuditEntry storage e = entries[id];
        return (e.decisionHash == decisionHash, e.shapHash == shapHash);
    }

    function getSubscriberEntries(bytes32 subscriberRef) external view
returns (uint256[] memory) {
        return subscriberEntries[subscriberRef];
    }

    function getStats() external view returns (uint256 total, uint256
approved, uint256 review, uint256 blocked) {
        return (entryCount, decisionCounts[0], decisionCounts[1],
decisionCounts[2]);
    }
}
```

Deterministic Rules and Decision Policy (Python) - rules.py

Six rules with severities, and the function that combines the model probability with the flags into the three-tier decision. A CRITICAL flag forces auto-block, a HIGH flag forces at least manual review, and each MEDIUM flag adds 0.10.

```
from typing import Dict, List, Tuple

MEDIUM_BOOST = 0.10
SWAP_COOLDOWN_DAYS = 7
AGENT_VELOCITY_LIMIT = 12
ACTIVE_SIM_MINUTES = 15
REMOTE_DISTANCE_KM = 50
AUTH_FAILURE_LIMIT = 5
REMOTE_CHANNELS = {1, 2}  # CALL_CENTRE, ONLINE


def evaluate_rules(features: Dict, imei_blacklisted: bool) -> List[Dict]:
    flags = []
    if imei_blacklisted:
        flags.append({"rule": "IMEI_BLACKLISTED", "severity": "CRITICAL",
                      "detail": "Target handset IMEI is on the
stolen/blocked device register"})
    if features["sim_age_days"] < SWAP_COOLDOWN_DAYS:
        flags.append({"rule": "SWAP_COOLDOWN", "severity": "HIGH",
                      "detail": f"Current SIM issued
{int(features['sim_age_days'])} day(s) ago "
                                f"(< {SWAP_COOLDOWN_DAYS}-day cooldown)"})
    if features["agent_swaps_24h"] >= AGENT_VELOCITY_LIMIT:
        flags.append({"rule": "AGENT_VELOCITY", "severity": "HIGH",
                      "detail": f"Agent processed
{int(features['agent_swaps_24h'])} swaps in 24 h "
                                f"(limit {AGENT_VELOCITY_LIMIT})"})
    if (features["old_sim_last_seen_min"] < ACTIVE_SIM_MINUTES
            and features["distance_from_home_km"] > REMOTE_DISTANCE_KM):
        flags.append({"rule": "ACTIVE_SIM_REMOTE_REQUEST", "severity":
"HIGH",
                      "detail": "Existing SIM is active on the network while
the swap is requested "
                                f"{features['distance_from_home_km']:.0f} km
away"})
    if features["kyc_biometric_verified"] == 0 and
int(features["channel_code"]) in REMOTE_CHANNELS:
        flags.append({"rule": "WEAK_KYC_REMOTE_CHANNEL", "severity":
"MEDIUM",
                      "detail": "Remote channel without biometric KYC"})
    if features["auth_failures_7d"] >= AUTH_FAILURE_LIMIT:
        flags.append({"rule": "AUTH_FAILURE_SPIKE", "severity": "MEDIUM",
                      "detail": f"{int(features['auth_failures_7d'])} failed
authentications in 7 days"})
    return flags


def decide(ml_probability: float, flags: List[Dict], t_review: float,
t_block: float) -> Tuple[float, str]:
    """Combine the ML probability with rule flags into (risk_score,
decision)."""
    severities = {f["severity"] for f in flags}
    n_medium = sum(1 for f in flags if f["severity"] == "MEDIUM")
    risk = min(1.0, ml_probability + MEDIUM_BOOST * n_medium)

    if "CRITICAL" in severities or risk >= t_block:
        decision = "AUTO_BLOCK"
    elif "HIGH" in severities or risk >= t_review:
        decision = "MANUAL_REVIEW"
    else:
```

```
        decision = "AUTO_APPROVE"
    return round(risk, 4), decision
```

Canonical Hashing and Pseudonymisation (Python) - chain.py

Records are serialised as canonical JSON (sorted keys, no whitespace, fixed rounding) and hashed with SHA-256. The SHAP hash covers the contributions and the feature values they explain. The subscriber reference is an HMAC of the mobile number.

```
def canonical_json(obj) -> bytes:
    return json.dumps(obj, sort_keys=True, separators=(",", ":"),
ensure_ascii=True).encode("utf-8")

def sha256_hex(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()

def pseudonymise(identifier: str) -> str:
    """HMAC-SHA256 pseudonym (hex). The MSISDN itself never leaves the
operator."""
    return hmac.new(OPERATOR_HMAC_KEY, identifier.encode(),
hashlib.sha256).hexdigest()

def decision_record(r: Dict) -> Dict:
    """Subset of a stored decision that is covered by the on-chain decision
hash."""
    return {
        "request_id": r["request_id"],
        "subscriber_ref": r["subscriber_ref"],
        "decision": r["decision"],
        "risk_score": round(float(r["risk_score"]), 4),
        "ml_probability": round(float(r["ml_probability"]), 6),
        "rule_flags": sorted(r["rule_flags"]),
        "model_version": r["model_version"],
        "timestamp": r["timestamp"],
    }

def shap_record(r: Dict) -> Dict:
    """Explanation covered by the on-chain SHAP hash: the numeric SHAP
vector plus
    the feature values it explains, so the readable reasons can be
regenerated."""
    return {
        "base_value": round(float(r["base_value"]), 6),
        "contributions": {k: round(float(v), 6) for k, v in
sorted(r["contributions"].items())},
        "features": {k: round(float(v), 6) for k, v in
sorted(r["features"].items())},
    }

def compute_hashes(r: Dict) -> Dict:
    return {
        "decision_hash": sha256_hex(canonical_json(decision_record(r))),
        "shap_hash": sha256_hex(canonical_json(shap_record(r))),
    }
```

Edge Telemetry Signing and Verification (Python) - telemetry.py

The gateway signs the packet with HMAC-SHA256 over a language-neutral canonical encoding. The engine checks the device, signature, freshness, nonce and plausibility before any model inference.

```
def _normalise(obj):
    """Make the encoding language-neutral: a JSON round-trip through
JavaScript turns 12.0 into 12,
    so integral floats are canonicalised to integers on both the signing and
verifying side."""
    if isinstance(obj, dict):
        return {k: _normalise(v) for k, v in obj.items()}
    if isinstance(obj, (list, tuple)):
        return [_normalise(v) for v in obj]
    if isinstance(obj, float) and obj.is_integer():
        return int(obj)
    return obj

def _canonical(obj) -> bytes:
    return json.dumps(_normalise(obj), sort_keys=True, separators=(",",
":")).encode()

def luhn_valid(number: str) -> bool:
    if not number.isdigit() or len(number) != 15:
        return False
    total = 0
    for i, ch in enumerate(reversed(number)):
        d = int(ch)
        if i % 2 == 1:
            d *= 2
            if d > 9:
                d -= 9
        total += d
    return total % 10 == 0

def sign_envelope(device_id: str, payload: Dict, ts: Optional[float] = None,
                  nonce: Optional[str] = None, key: Optional[bytes] = None)
-> Dict:
    ts = time.time() if ts is None else ts
    nonce = nonce or uuid.uuid4().hex
    key = key or EDGE_DEVICE_KEYS[device_id]
    payload = copy.deepcopy(payload)       # the envelope must not alias the
caller's dict
    body = {"device_id": device_id, "nonce": nonce, "ts": round(ts, 3),
"payload": payload}
    sig = hmac.new(key, _canonical(body), hashlib.sha256).hexdigest()
    return body | {"signature": sig}

def verify_envelope(env: Dict, seen_nonces: Set[str], now: Optional[float] =
None) -> Tuple[bool, str]:
    """Return (accepted, reason). Does NOT record the nonce; the caller does
so
    after the request has been fully processed."""
    now = time.time() if now is None else now
    try:
        device_id, nonce, ts, payload, sig = (env["device_id"],
env["nonce"], env["ts"],
```

```
                                                env["payload"],
env["signature"])
    except KeyError as exc:
        return False, f"malformed envelope (missing {exc.args[0]})"
    key = EDGE_DEVICE_KEYS.get(device_id)
    if key is None:
        return False, "unknown edge device"
    body = {"device_id": device_id, "nonce": nonce, "ts": ts, "payload":
payload}
    expected = hmac.new(key, _canonical(body), hashlib.sha256).hexdigest()
    if not hmac.compare_digest(expected, str(sig)):
        return False, "signature mismatch (payload altered or wrong key)"
    if abs(now - float(ts)) > TELEMETRY_MAX_AGE_S:
        return False, "stale telemetry (outside freshness window)"
    if nonce in seen_nonces:
        return False, "replayed nonce"
    imei = str(payload.get("imei", ""))
    if not luhn_valid(imei):
        return False, "implausible IMEI (checksum failed)"
    if not -120 <= float(payload.get("rssi_dbm", -70)) <= -30:
        return False, "implausible RSSI"
    for f in ("old_sim_last_seen_min", "distance_from_home_km",
"cell_changes_24h", "device_age_days"):
        if float(payload.get(f, 0)) < 0:
            return False, f"negative value for {f}"
    return True, "ok"
```

Risk Engine Evaluation Pipeline (Python) - engine.py

One request from signed telemetry to anchored decision: verify, build features, score and explain, apply rules and policy, hash, anchor on the blockchain and store.

```
def evaluate(self, req: Dict) -> Dict:
    t_all = time.perf_counter()
    lat: Dict[str, float] = {}

    # 1. Telemetry integrity ---------------------------------------------
--
    t = time.perf_counter()
    env = req.get("telemetry")
    if not env:
        raise TelemetryRejected("missing signed edge telemetry")
    seen = {env.get("nonce")} if
self.store.nonce_seen(str(env.get("nonce"))) else set()
    ok, reason = verify_envelope(env, seen)
    if not ok:
        self.rejections.append({"time": utc_now(), "device":
env.get("device_id"), "reason": reason})
        self.rejections = self.rejections[-100:]
        raise TelemetryRejected(reason)
    payload = env["payload"]
    lat["telemetry_ms"] = (time.perf_counter() - t) * 1000

    request_id = req.get("request_id") or "REQ-" +
uuid.uuid4().hex[:10].upper()
    if self.store.get_decision(request_id):
        raise DuplicateRequest(request_id)
```

```
    # 2. Features, ML score and SHAP ----------------------------------------
---
    features = self._features(req, payload)
    t = time.perf_counter()
    sc = explain.score(features)
    lat["ml_shap_ms"] = (time.perf_counter() - t) * 1000

    # 3. Deterministic rules + blacklist ------------------------------------
---
    t = time.perf_counter()
    blacklisted = self.store.is_blacklisted(str(payload["imei"]))
    flags = evaluate_rules(features, blacklisted)
    risk, decision = decide(sc["probability"], flags, self.t_review,
self.t_block)
    reasons = explain.top_reasons(features, sc["contributions"], k=8)
    lat["rules_ms"] = (time.perf_counter() - t) * 1000

    # 4. Canonical record and hashes ----------------------------------------
---
    t = time.perf_counter()
    ts = utc_now()
    record = {
        "request_id": request_id,
        "subscriber_ref": pseudonymise(req["msisdn"]),
        "decision": decision,
        "risk_score": risk,
        "ml_probability": sc["probability"],
        "rule_flags": [f["rule"] for f in flags],
        "model_version": sc["model_version"],
        "timestamp": ts,
        "base_value": sc["base_value"],
        "contributions": sc["contributions"],
        "features": features,
    }
    hashes = compute_hashes(record)
    lat["hashing_ms"] = (time.perf_counter() - t) * 1000

    # 5. Anchor on the blockchain -------------------------------------------
---
    with self._chain_lock:
        anchor = self.chain.log_decision(record["subscriber_ref"], decision,
risk,
                                         hashes["decision_hash"],
hashes["shap_hash"])
    lat["chain_ms"] = anchor["chain_ms"]

    # 6. Persist the off-chain record ---------------------------------------
---
    t = time.perf_counter()
    self.store.remember_nonce(env["nonce"], env["device_id"],
float(env["ts"]))
    row = {
        "request_id": request_id,
        "entry_id": anchor["entry_id"],
        "subscriber_ref": record["subscriber_ref"],
        "msisdn_masked": mask_msisdn(req["msisdn"]),
        "channel": req["channel"],
        "agent_ref": hashlib.sha256(str(req.get("agent_id",
"")).encode()).hexdigest()[:12],
        "decision": decision,
```

```
        "risk_score": risk,
        "ml_probability": sc["probability"],
        "rule_flags": record["rule_flags"],
        "rule_detail": flags,
        "model_version": sc["model_version"],
        "timestamp": ts,
        "base_value": sc["base_value"],
        "contributions": sc["contributions"],
        "features": features,
        "reasons": reasons,
        "summary": build_summary(decision, risk, reasons[:4], flags),
        "edge_device": env["device_id"],
        "decision_hash": hashes["decision_hash"],
        "shap_hash": hashes["shap_hash"],
        "tx_hash": anchor["tx_hash"],
        "block_number": anchor["block_number"],
        "gas_used": anchor["gas_used"],
        "latency_ms": {},
    }
    lat["db_ms"] = (time.perf_counter() - t) * 1000
    lat["total_ms"] = (time.perf_counter() - t_all) * 1000
    row["latency_ms"] = {k: round(v, 2) for k, v in lat.items()}
    self.store.insert_decision(row)
    return row
```

Integrity Audit (Python) - engine.py

The audit reads every on-chain entry, recomputes the hashes of the matching off-chain record and reports edits, deletions and records that were never anchored.

```
def audit(self) -> Dict:
    """Verify every off-chain record against the blockchain."""
    t0 = time.perf_counter()
    n_chain = self.chain.entry_count()
    rows = self.store.all_decisions()
    by_entry = {r["entry_id"]: r for r in rows if r["entry_id"] is not None}
    findings, checked, ok_count = [], 0, 0
    for eid in range(n_chain):
        row = by_entry.get(eid)
        if row is None:
            findings.append({"entry_id": eid, "request_id": None, "reason":
                             "on-chain entry has no off-chain record (record
deleted or never stored)"})
            continue
        checked += 1
        res = self.chain.verify_record(self.hashable(row), eid)
        if res["ok"]:
            ok_count += 1
        else:
            findings.append({"entry_id": eid, "request_id":
row["request_id"], "reason": res["reason"]})
    for r in rows:
        if r["entry_id"] is None or r["entry_id"] >= n_chain:
            findings.append({"entry_id": r["entry_id"], "request_id":
r["request_id"],
                             "reason": "off-chain record is not anchored on
the blockchain"})
    return {"on_chain_entries": n_chain, "off_chain_records": len(rows),
"verified": ok_count,
            "tampered": len(findings), "integrity_ok": not findings,
"findings": findings,
            "audit_seconds": round(time.perf_counter() - t0, 3)}
```

Policy Calibration (Python) - train_model.py

The review and block thresholds are chosen on the validation set by minimising an explicit operating cost under an analyst capacity limit.

```
def calibrate_policy(y, prob, crit, high, n_med):
    risk = np.minimum(1.0, prob + MEDIUM_BOOST * n_med)
    best = None
    for t_review in np.round(np.arange(0.02, 0.61, 0.01), 2):
        for t_block in np.round(np.arange(t_review + 0.05, 0.99, 0.01), 2):
            block = crit | (risk >= t_block)
            review = ~block & (high | (risk >= t_review))
            approve = ~block & ~review
            if review.mean() > REVIEW_CAPACITY:
                continue
            cost = (COST_MISSED_FRAUD * np.sum(approve & (y == 1))
                    + COST_FALSE_BLOCK * np.sum(block & (y == 0))
                    + COST_REVIEW * np.sum(review))
            if best is None or cost < best[0]:
                best = (cost, float(t_review), float(t_block))
    return {"t_review": best[1], "t_block": best[2], "validation_cost":
float(best[0])}
```

SCREENSHOTS

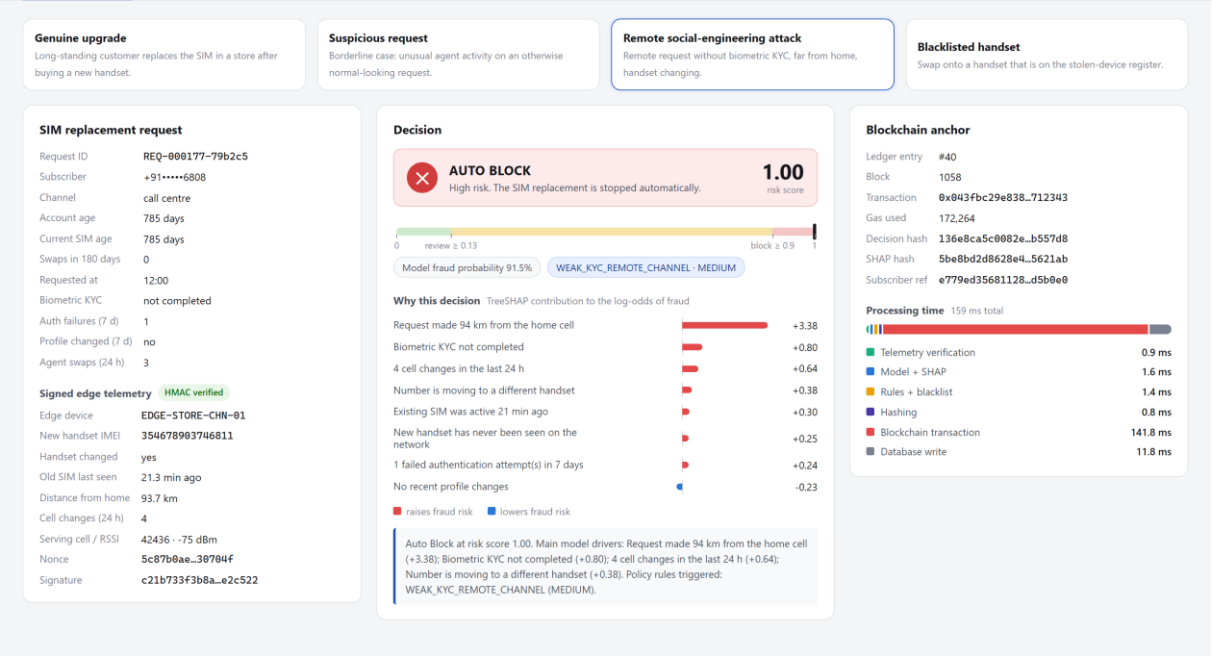


Fig. A.1 – Operations console, Evaluate request view: a remote social-engineering attack is auto-blocked

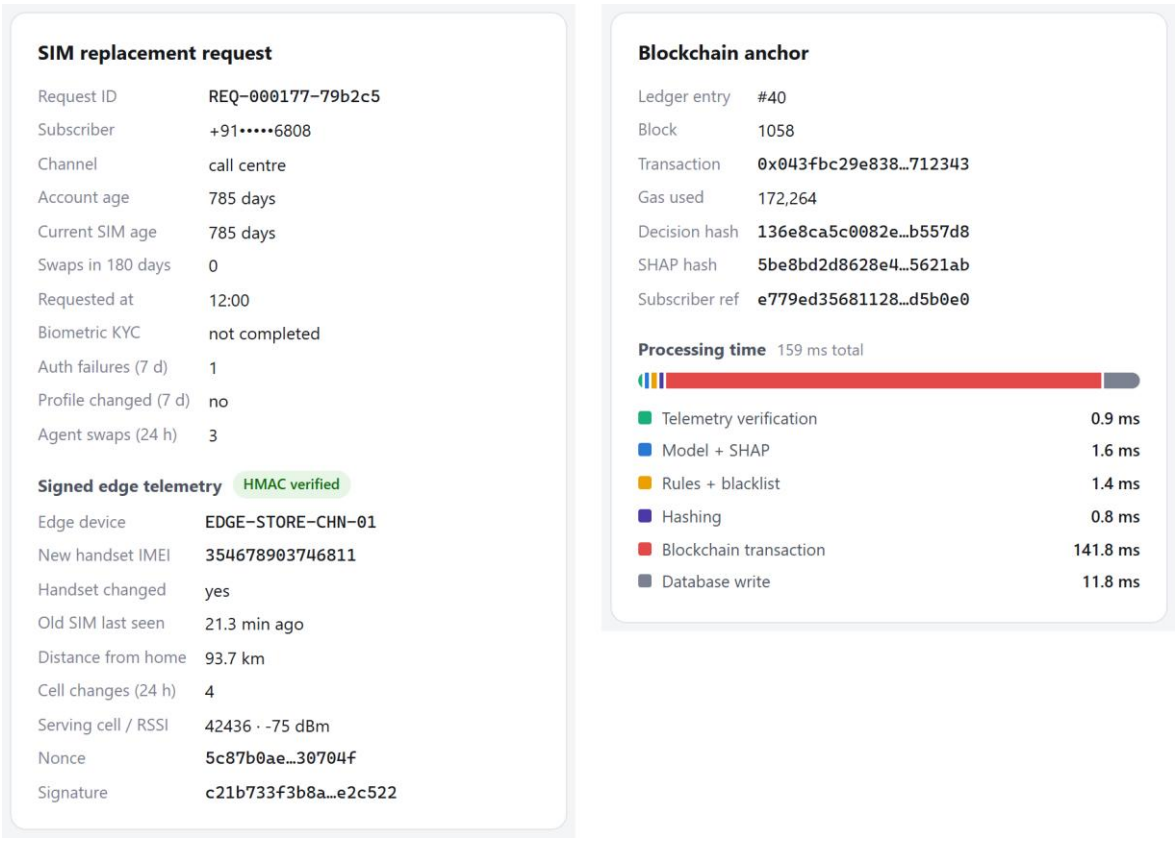


Fig. A.2 – Request with signed edge telemetry (left) and blockchain anchor with processing time (right)



Fig. A.3 – Decision panel: genuine upgrade, auto-approved

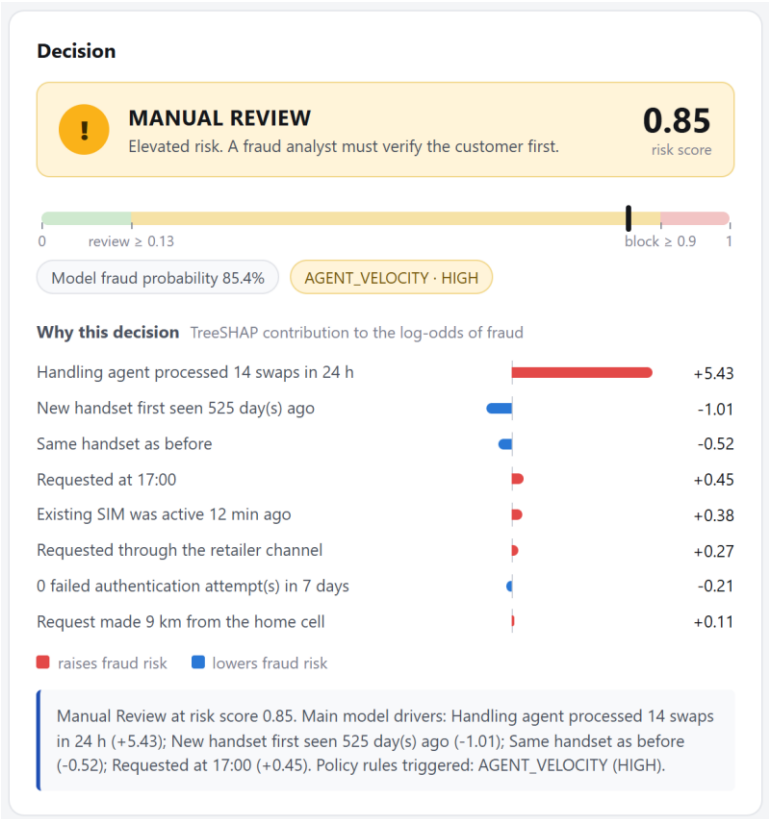


Fig. A.4 – Decision panel: suspicious request routed to manual review

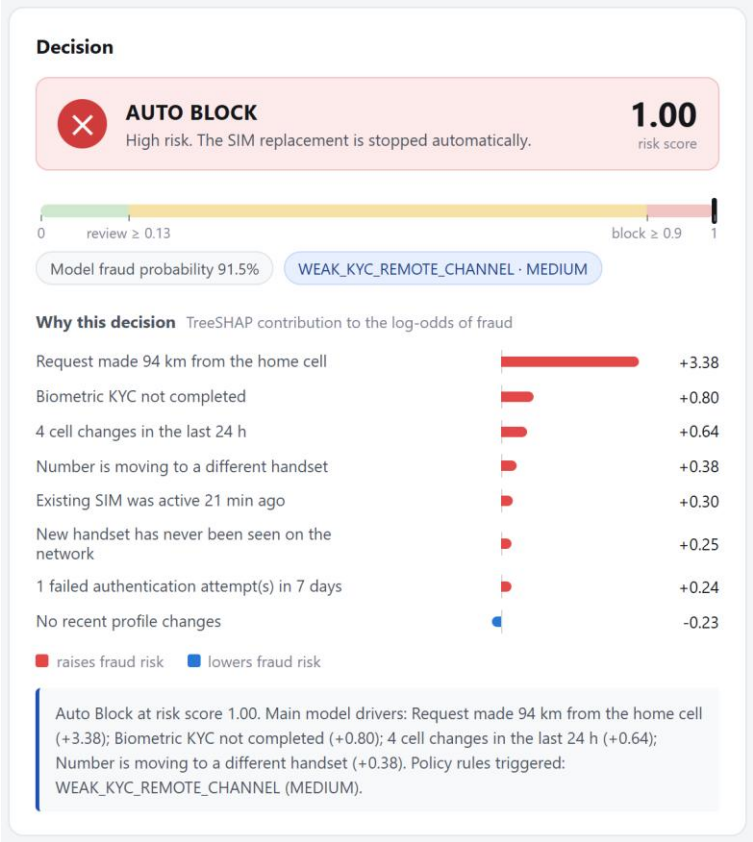


Fig. A.5 – Decision panel: remote social-engineering attack, auto-blocked

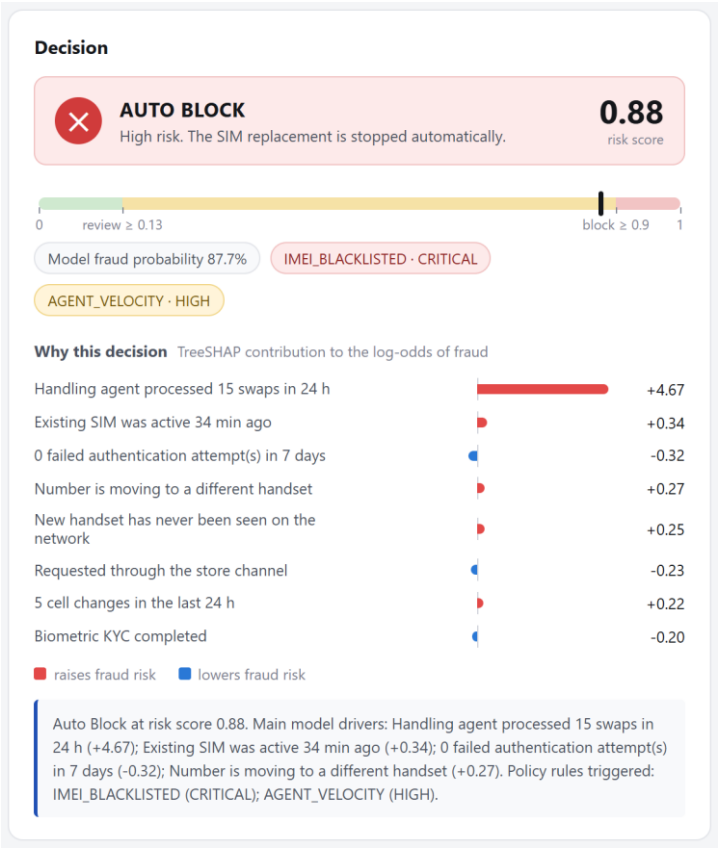


Fig. A.6 – Decision panel: blacklisted handset forces auto-block

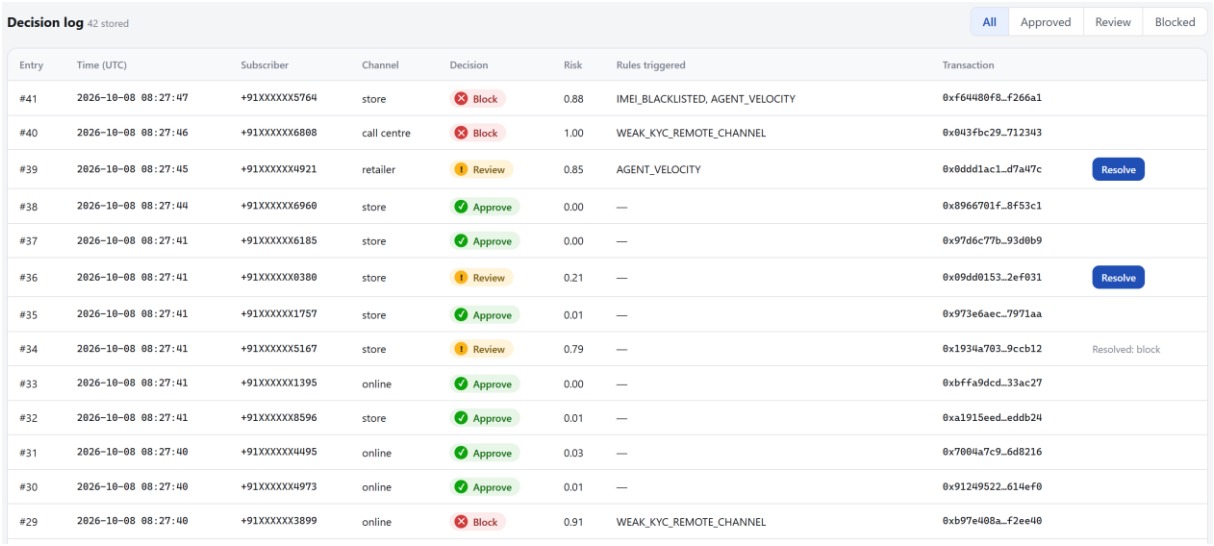


Fig. A.7 – Decision log with the Resolve action for manual reviews

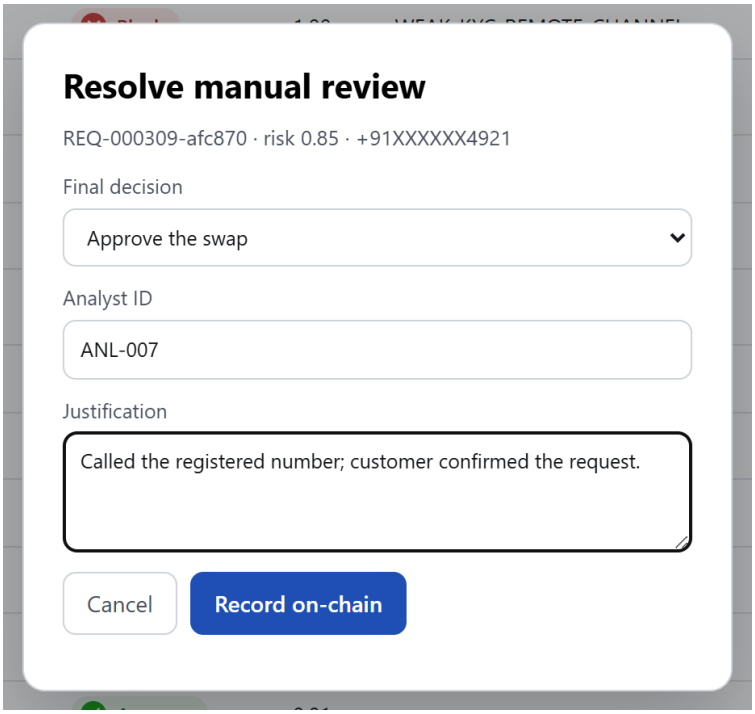


Fig. A.8 – Analyst resolves a manual review; the outcome is recorded on-chain

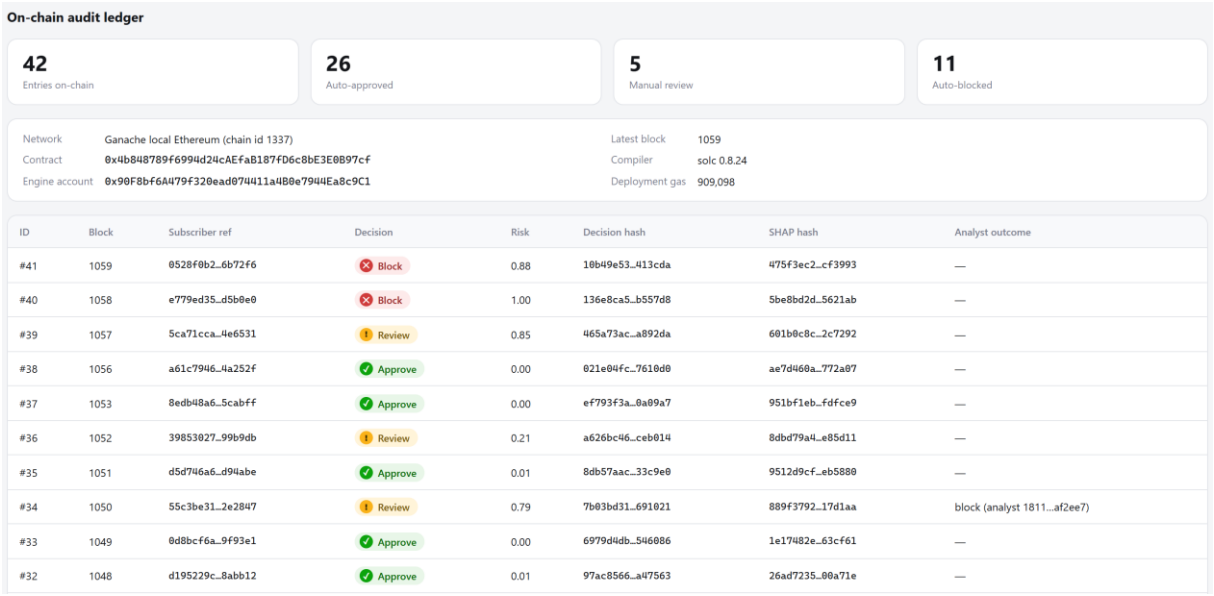


Fig. A.9 – Blockchain ledger: contract details, counters and latest entries

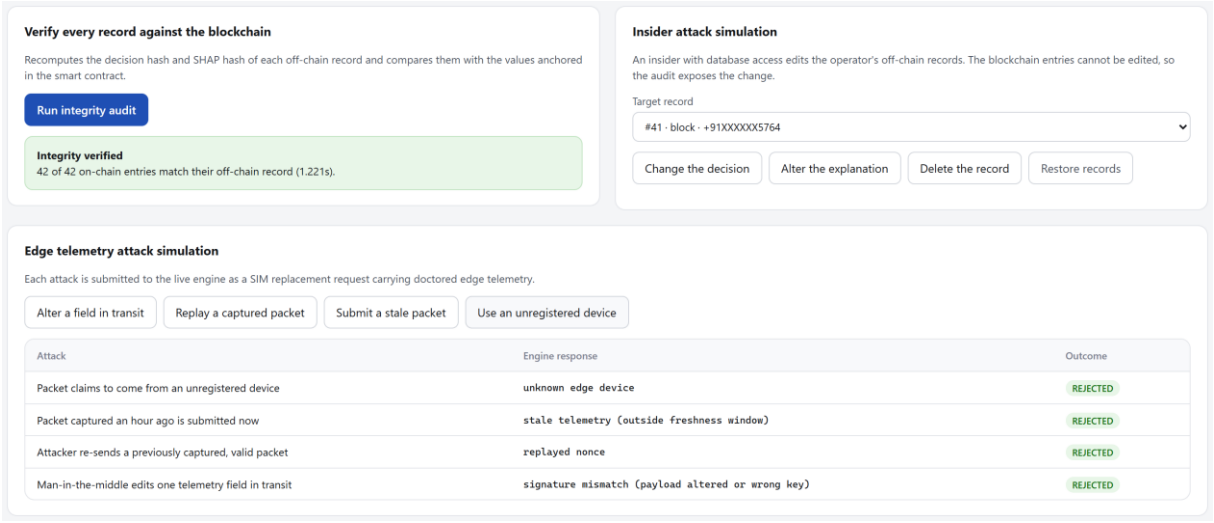


Fig. A.10 – Integrity audit passed; four telemetry attacks rejected

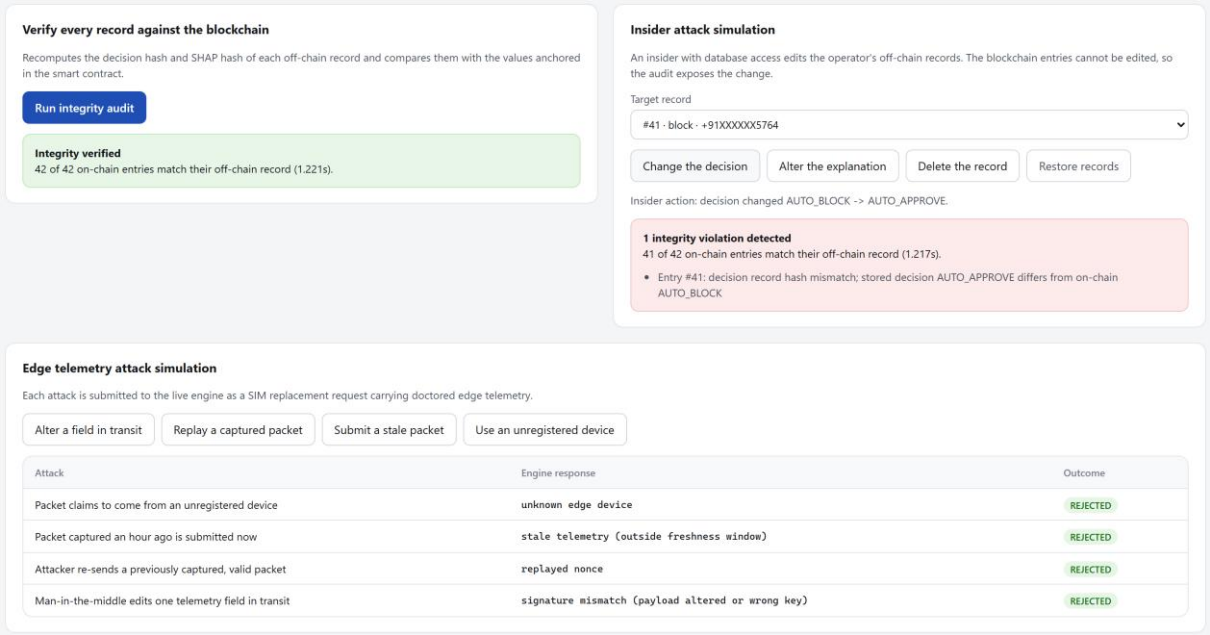


Fig. A.11 – Integrity audit exposes an insider's edit of a stored decision

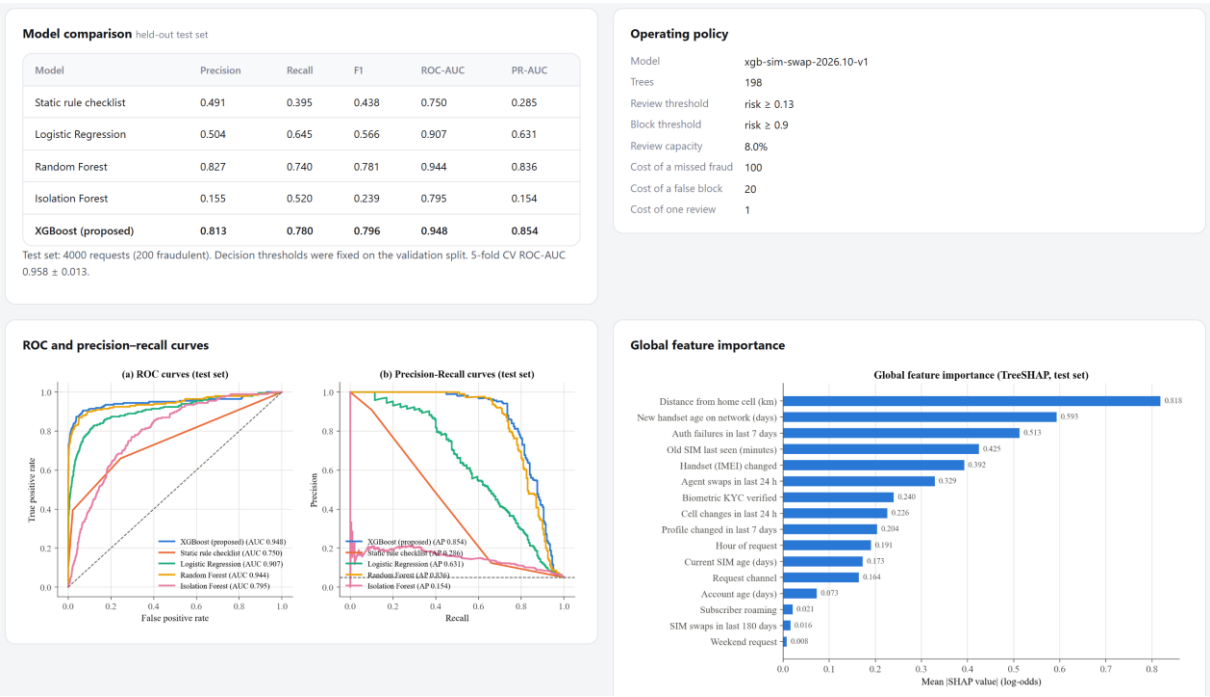


Fig. A.12 – Model comparison and operating policy

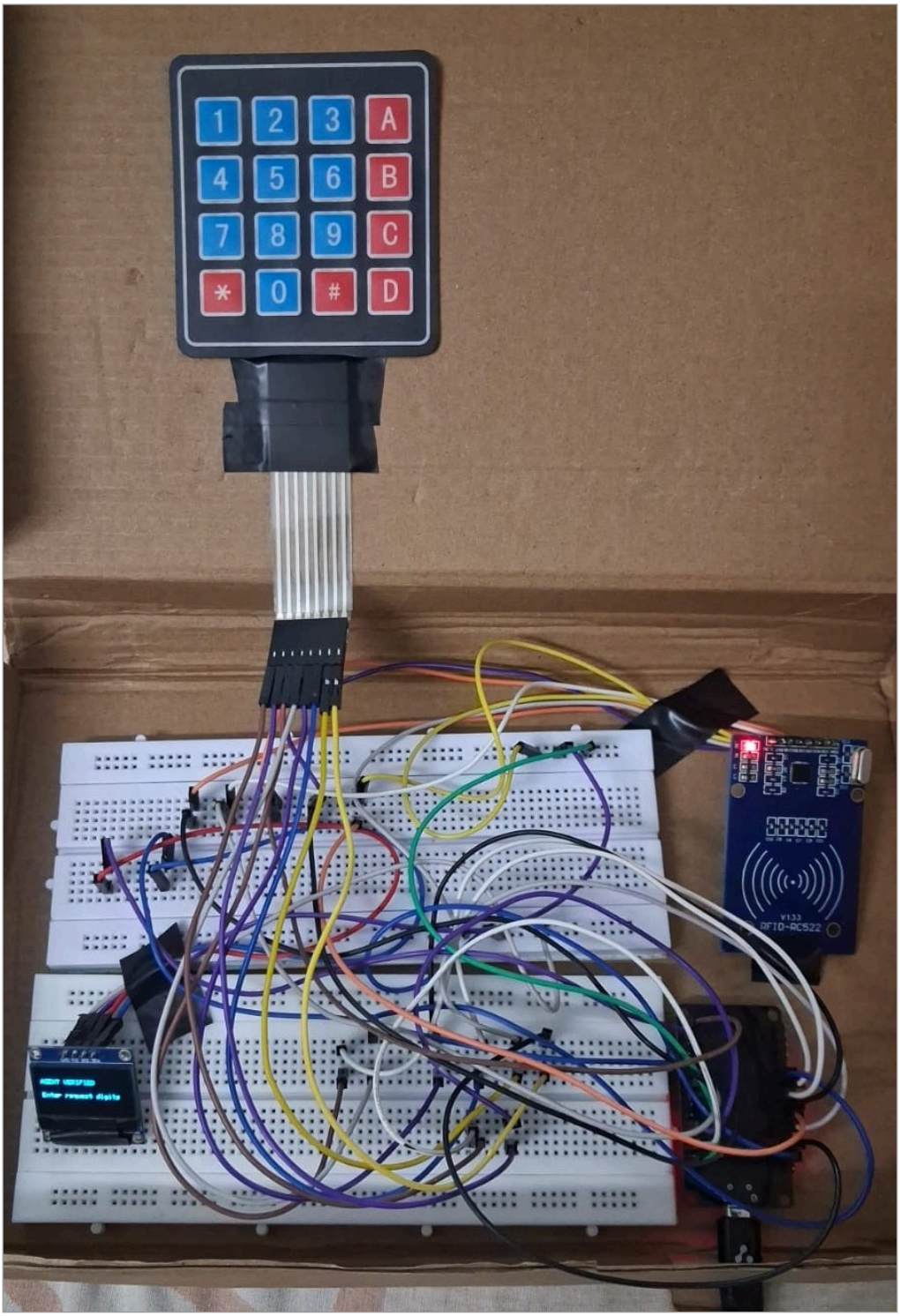


Fig. A.13 – Hardware prototype of the agent-verification terminal

Keypad (top), RC522 RFID reader (right), OLED display (left) and microcontroller board (bottom right) wired on solderless breadboards; the display reads "AGENT VERIFIED / Enter request digits".